سُبْحَانَكَ لاَ عِلْمَ لَنَا إِلاَّ مَا عَلَّمْتَنَا ۖ إِنَّكَ أَنْتَ الْعَلِيمُ الْحَكِيمُ

Dedications

In the name of Allah, the Most Gracious, the Most Merciful

To our beloved parents,

whose unwavering support, endless sacrifices, and constant prayers
have been the foundation of our success.

To our teachers and mentors,
who guided us with knowledge and wisdom
throughout our academic journey.

To the team at Novation City,
who welcomed us with open arms and provided
invaluable learning opportunities.

To our families and friends,
whose encouragement and understanding
made this achievement possible.

With deep gratitude and appreciation.

Acknowledgments

I would like to express my sincere gratitude to all those who contributed to the success of this professional internship experience, including academic supervisors, industry mentors, and institutional staff whose guidance, feedback, and practical support were essential to achieving the project objectives.

First, I extend my deepest appreciation to my academic supervisor, **Mr. Ahmed Bin Ayed**, for their invaluable guidance and continuous support throughout this project. Their expertise was instrumental in ensuring academic rigor and alignment with university requirements.

I am profoundly grateful to the Novation City team who welcomed me into their innovative ecosystem. Special thanks to my professional supervisor, **Mr. Mohamed Chrifa**, for their professional insights and technical guidance in Industry 4.0 implementation and digital transformation consulting.

My sincere appreciation goes to Novation City competitiveness cluster for entrusting me with developing the **IoT-REAP (Internet of Things - Remote Equipment Access Platform)**. Building this secure industrial remote operations platform—integrating Proxmox VE virtualization, Apache Guacamole remote desktop, MQTT-based IoT robot control, and AI-powered demand forecasting—has been an invaluable experience for my professional growth in full-stack development and industrial systems integration.

I extend gratitude to EPI Digital School's administration and faculty for providing quality education and academic resources essential for understanding enterprise-grade web application development, security architecture, and modern software engineering practices.

Finally, I am grateful to my family and friends for their unwavering support and encouragement throughout this journey.

Contents

List of Figures

List of Tables

Chapter 1

Conceptual Study

Introduction

This chapter presents the conceptual study of the IoT-REAP platform. It describes the adopted software architecture, models the system through use case diagrams and representative sequence and activity diagrams, and presents the class diagram used to guide realization.

1 Adopted Software Architecture

1.1 Architectural Pattern Overview

The Model-View-Controller (MVC) architectural pattern is a foundational design pattern in software development that promotes separation of concerns, maintainability, and scalability [**?**]. MVC divides applications into three interconnected components:

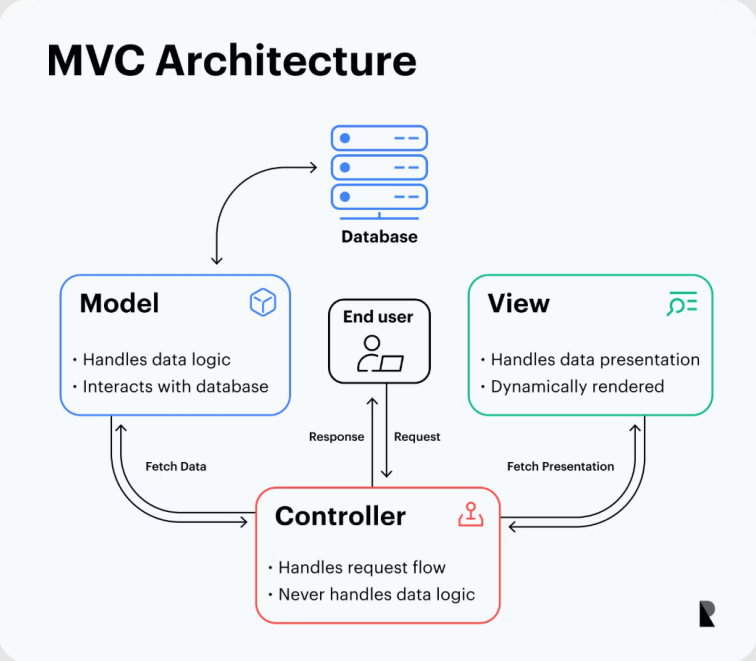

Figure 1.1: MVC Architectural Pattern: Component Interaction Flow

Model Component: Represents data structures, business logic, and data management rules. Responsible for data persistence, retrieval, validation, and enforcing business constraints independently of the user interface.

View Component: Handles presentation layer and user interface rendering. Displays data in appropriate formats and captures user input without containing business logic.

Controller Component: Manages data flow between Models and Views, orchestrating

application logic. Receives user requests, interacts with Models, applies business rules, and selects appropriate Views for rendering responses.

This architectural separation enables independent development, testing, and modification of each component, reducing code coupling and improving maintainability.

1.2 MVC Layers in IoT-REAP Context

The Model Layer encapsulates data structures, database interactions, and business rules. IoT-REAP Models define database structure for users, VM sessions, robots, telemetry, and audit logs. Models implement entity relationships, provide CRUD operations, and enforce data integrity through Eloquent ORM patterns.

The View Layer in IoT-REAP is implemented as a React 18 frontend with TypeScript. Key principles include component-based architecture, typed props interfaces, custom hooks for data fetching, and Zustand for state management. Views render dashboards, session interfaces, robot control panels, and admin consoles.

The Controller Layer orchestrates application logic between Models and Services. IoT-REAP Controllers remain thin—validating input via FormRequests, delegating to Services for business logic, and returning JSON responses via API Resources. Controllers handle authentication/authorization via middleware and manage error responses.

The Service Layer contains all business logic. Services like `VMProvisioningService`, `GuacamoleClient`, `MqttService`, and `RiskScoreEngine` encapsulate complex operations, enabling testability and reuse.

The Repository Layer handles all database queries. Repositories like `VMSessionRepository` and `UserRepository` abstract Eloquent operations, returning Models or Collections—never raw arrays.

Table 1.1: IoT-REAP Platform Core Controllers and Responsibilities

Controller	Primary Responsibilities
AuthController	User authentication, registration, password reset, session management
VMSessionController	Session creation, extension, termination; delegates to VMProvisioningService
VMTemplateController	Template listing, admin CRUD operations for VM templates
ProxmoxNodeController	Node stats, VM listing, VM control (start/stop/reboot)
RobotSessionController	Robot reservation, command dispatch via MqttService
SecurityEventController	Security event listing, active sessions with risk scores
AdminController	User quota management, activity logs, system configuration

1.3 MVC Benefits for IoT-REAP Platform

The MVC architectural pattern provides critical advantages:

Separation of Concerns: Database specialists focus on Model optimization, frontend developers concentrate on React components, backend developers work on Services and Controllers, enabling parallel development without conflicts.

Maintainability: Database schema changes don't require React modifications; UI redesigns don't affect business logic; clear organization enables rapid developer onboarding.

Testability: Models and Services can be unit tested independently; Controllers tested through feature tests with mocked external APIs (Proxmox, Guacamole); React components tested with React Testing Library.

Reusability: Services reused across multiple Controllers (e.g., `ProxmoxClient` used by both `VMSessionController` and `ProxmoxNodeController`); React hooks shared across pages.

Scalability: Independent layer scaling based on performance requirements; AI microservice scales separately from Laravel backend.

Table 1.2: How MVC Architecture Supports IoT-REAP Platform Requirements

Requirement	**MVC Architectural Support**
VM Provisioning	Service layer encapsulates Proxmox API; Controllers orchestrate workflow
Remote Desktop	GuacamoleClient service handles connection lifecycle; Views embed viewer
Robot Control	MqttService publishes commands; WebSocket broadcasts telemetry to Views
AI Forecasting	External microservice called via HTTP; results cached and served to Views
Zero-Trust Security	Middleware layer computes risk scores; Controllers enforce access policies
Audit Logging	Observer pattern on Models; AuditLogger service maintains hash chain
Real-Time Updates	Events broadcast via Laravel Echo; React Views subscribe via WebSocket

The MVC pattern, extended with Service and Repository layers, provides the essential structural foundation enabling the IoT-REAP platform to meet all functional and non-functional requirements while maintaining high code quality and long-term maintainability.

2 Use Case Diagram

After identifying actors and requirements, the conceptual analysis of the platform can be refined through use case modeling. The implemented application suggests one general use case view and several actor-specific refinements. The general model places the platform at the center and connects the four user actors (Guest, Engineer, Teacher, Administrator) as well as the external Stripe actor to the services they invoke: public discovery, learning progression, content authoring, infrastructure access, commerce, and administrative governance.

2.1 General Use Case Diagram

The general use case diagram groups system behavior into four principal subsystems:

In this model, the Engineer extends the Guest by inheriting public consultation capabilities and adding authenticated learning and virtual laboratory operations. The Teacher extends the Engineer's authenticated access with content authoring and instructional management. The Administrator occupies the supervisory position by controlling both business governance and

infrastructure operations. Stripe is modeled as a standalone external actor that interacts with the platform through checkout, webhook callbacks, refunds, and payout-related flows. Figure 1.2 presents the general use case diagram of the IoT-REAP platform, showing the four user actors plus Stripe and their principal interactions with the system subsystems.

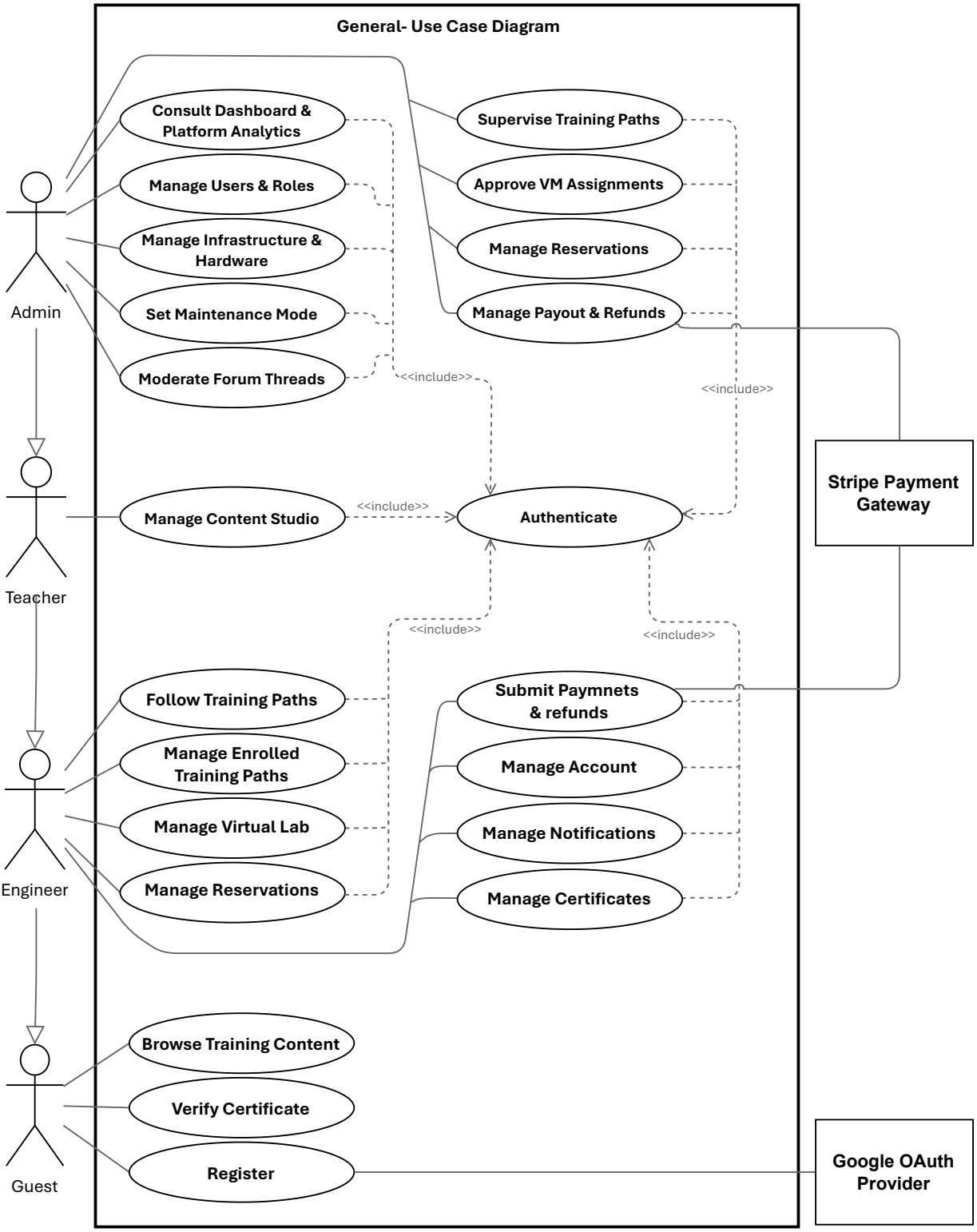

Figure 1.2: General Use Case Diagram

2.2 Refinement and Specification of Use Cases

The major use cases can be refined through representative specifications aligned with the actual implemented routes and domain behaviors. Detailed, actor-specific use case specifications (Guest, Engineer, Teacher, Administrator) are presented in Chapter 3 along with sequence diagrams and concrete implementations; they are not inlined here to avoid duplication between chapters and to keep the conceptual chapter focused on models and high-level diagrams.

2.2.1 Refinement of the Use Case Manage Users & Roles

Figure 1.3 presents the refinement of the Use Case Manage Users & Roles for the Administrator actor. This use case includes the following sub-use cases:

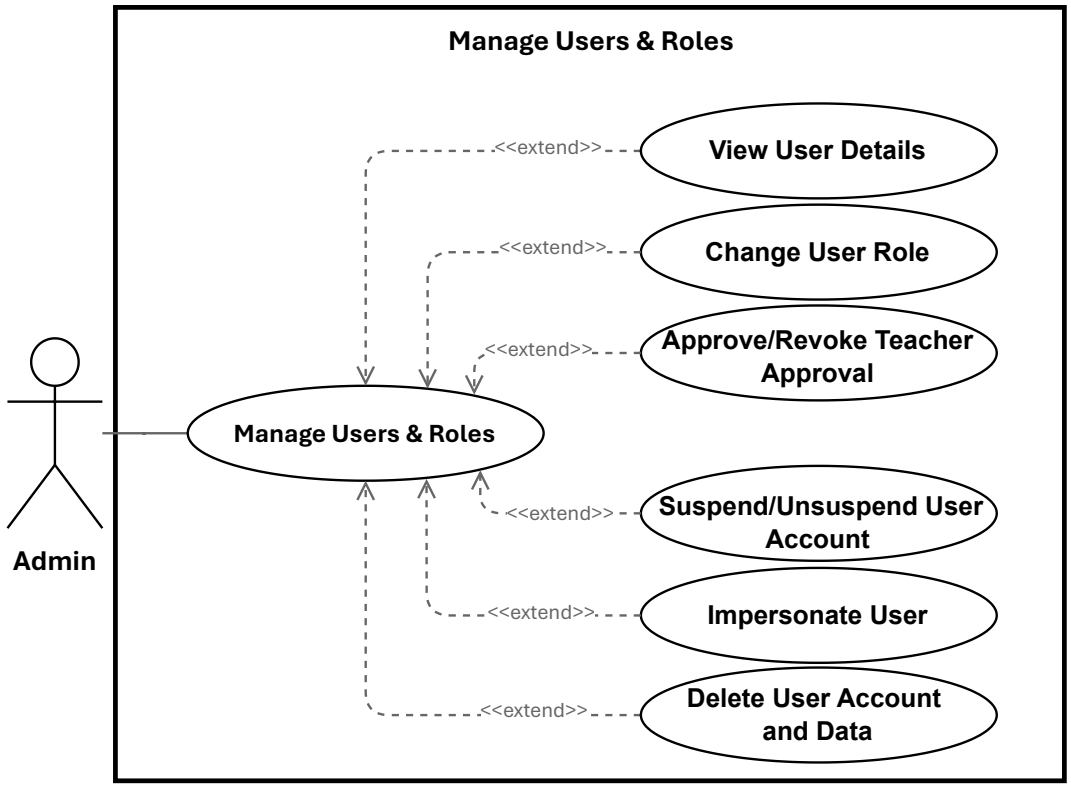

Figure 1.3: Use Case Manage Users & Roles

2.2.1.1 Specification of the Use Case Change User Role

Table 1.3: Use Case Specification: Change User Role

Title	Change User Role
Summary	Administrator updates a user's role (engineer, teacher, admin). Assigning `teacher` also records teacher-approval metadata; assigning a non-teacher clears any teacher-approval metadata.
Actor	Administrator
Precondition	Administrator is authenticated and has admin privileges. Target user exists. Requested role is one of: `engineer`, `teacher`, `admin`.
Post-condition	The user's role is persisted. If role = `teacher`, `teacher_approved_at` and `teacher_approved_by` are set; otherwise those fields are cleared. The change is logged and the updated user is returned.
Basic Scenario	1. Administrator opens the user management list or the target user's detail page. 2. Administrator selects a new role and submits the change. 3. System validates request authorization and that the role value is allowed. 4. System verifies the administrator is not attempting to change their own role. 5. System prepares the update payload (including teacher-approval metadata when applicable). 6. System persists the update to the database. 7. System logs the role change for audit purposes. 8. System returns the updated user record to the admin UI and displays confirmation.
Error Scenario	1. If the administrator attempts to change their own role, the system rejects the request and returns an error. 2. If the submitted role is invalid, the system returns a validation error and no update is made. 3. If persistence or service errors occur, the system aborts the update, logs the failure, and returns an error. 4. If the administrator is not authorized, the system returns an authorization error (403).

2.2.1.2 Specification of the Use Case Approve/Revoke Teacher Approval

Table 1.4: Use Case Specification: Approve/Revoke Teacher Approval

Title	Approve/Revoke Teacher Approval
Summary	Administrator approves a teacher account to mark it as authorized, or revokes an existing teacher approval to remove that authorization. Approving records teacher-approval metadata; revoking clears it.
Actor	Administrator
Precondition	Administrator is authenticated with admin privileges. Target user exists and has the `teacher` role.
Post-condition	If approved, `teacher_approved_at` and `teacher_approved_by` are set. If revoked, those fields are cleared. The change is logged and the updated user record is returned.
Basic Scenario	1. Administrator opens the teacher account details in the user management section. 2. Administrator selects *Approve Teacher* or *Revoke Teacher Approval*. 3. System validates that the target account is a teacher and that the request is authorized. 4. System prepares the update payload for the selected action. 5. System persists the approval change to the database. 6. System logs the action for audit purposes. 7. System returns the updated user record to the admin UI and displays confirmation.
Error Scenario	1. If the target user is not a teacher, the system returns a validation error and no change is made. 2. If the administrator is not authorized, the system returns an authorization error (403). 3. If persistence fails, the system aborts the update, logs the failure, and returns an error.

2.2.1.3 Specification of the Use Case Suspend/Unsuspend User Account

Table 1.5: Use Case Specification: Suspend / Unsuspend User Account

Title	Suspend / Unsuspend User Account
Summary	Administrator temporarily blocks or restores user access. May specify a reason and optional expiration date for temporary suspensions. All actions are recorded for audit.
Actor	Administrator
Precondition	Administrator is authenticated and authorized to manage accounts. Target user exists.
Post-condition	User access status is updated (suspended or active). Metadata (reason, expiration) is recorded. Action is logged for audit. If suspended, user can no longer log in. If unsuspended, access is restored.
Basic Scenario	1. Administrator opens user management and selects an account. 2. Administrator chooses Suspend Account or Restore Access. 3. For suspension: Administrator optionally provides a reason and expiration date. 4. System validates administrator authorization. 5. System checks that target user exists and is eligible (e.g., not an administrator). 6. System updates account status and records metadata. 7. System blocks or restores access immediately. 8. System logs the action for audit. 9. System notifies user of status change (if configured). 10. Administrator receives confirmation and updated status is displayed.
Error Scenario	1. If administrator lacks permission, request is denied and authorization error is displayed. 2. If target user does not exist, system displays error and does not proceed. 3. If target is an administrator, system prevents suspension and displays error. 4. If action cannot be completed due to system errors, system rolls back changes, preserves account state, and displays error. 5. If notifications are configured but unavailable, core action is completed, but system flags notification failure for manual follow-up.

2.2.1.4 Specification of the Use Case Impersonate User

Table 1.6: Use Case Specification: Impersonate User

Title	Impersonate User
Summary	A privileged administrator or support agent temporarily assumes a target user's identity for troubleshooting. Impersonation is time-limited, auditable, and constrained by authorization rules.
Actor	Administrator
Precondition	Initiator is authenticated and authorized to impersonate.
Post-condition	A temporary impersonation session is created. Actions performed are applied as the target user but recorded with initiator metadata. Session start and end are logged.
Basic Scenario	1. Administrator initiates impersonation via UI or API, specifying target user ID and mode (read-only/full). 2. System validates and sanitizes request parameters. 3. System verifies the initiator's permission to impersonate the target. 4. System loads the target user record and verifies eligibility (e.g., active status, lower privilege). 5. System creates a time-limited session/token scoped to the target user, annotated with initiator metadata and mode constraints. 6. System logs an immutable audit event for impersonation start (including session ID, mode, and reason). 7. System returns the token or redirects the initiator into the target's session context; initiator acts within the session and actions execute as the target user but include initiator audit metadata. 8. Initiator ends impersonation, or token expires; system revokes the session/token. 9. System logs impersonation end details (session ID, initiator, target, timestamps).
Error Scenario	1. Unauthorized initiator: System returns HTTP 403 and logs a failed attempt. 2. Target user not found or inactive: System returns HTTP 404/400 and logs failure. 3. Business rule violation (e.g., target has equal/higher privilege): System returns HTTP 403 and logs reason. 4. Session or audit logging failure: Impersonation aborts, returns an error, and clears partial session state.

2.2.1.5 Specification of the Use Case Delete User Account and Data

Table 1.7: Use Case Specification: Delete User Account and Data

Title	Delete User Account and Data
Summary	Administrator removes a user account by anonymizing personal information and removing non-essential data. Payment records preserved for audit.
Actor	Administrator
Precondition	Administrator is authenticated and authorized. Target user exists and is not an administrator. Administrator is not deleting own account.
Post-condition	Account removed. Personal information anonymized. Credentials revoked. Non-essential data removed. Payment history preserved. Deletion audited.
Post-condition (Failure)	No changes made. System reports failure and provides guidance.
Trigger	Administrator confirms deletion via the admin UI.
Basic Scenario	1. Administrator locates target account. 2. Administrator selects user and chooses *Delete Account*. 3. System displays confirmation dialog with deletion details. 4. Administrator confirms deletion. 5. System verifies target is not an administrator and not self. 6. System anonymizes personal information. 7. System removes credentials and session tokens. 8. System removes non-essential data. 9. System preserves financial records. 10. System records deletion in audit logs. 11. System displays success and removes user from active list.
Error Scenario	1. If administrator lacks permission, system rejects request and displays authorization error. 2. If target user is an administrator or administrator attempts to delete own account, system prevents deletion and displays error. 3. If target user does not exist, system displays error and does not proceed. 4. If deletion fails due to system errors, system rolls back changes, preserves user record, and displays error. 5. If partial cleanup fails, system preserves core deletion but flags incomplete cleanup for manual review.

2.2.2 Refinement of the Use Case Manage Infrastructure and Hardware

Figure 1.4 presents the refinement of the Use Case Manage Infrastructure and Hardware for the Administrator actor. This use case includes the following sub-use cases:

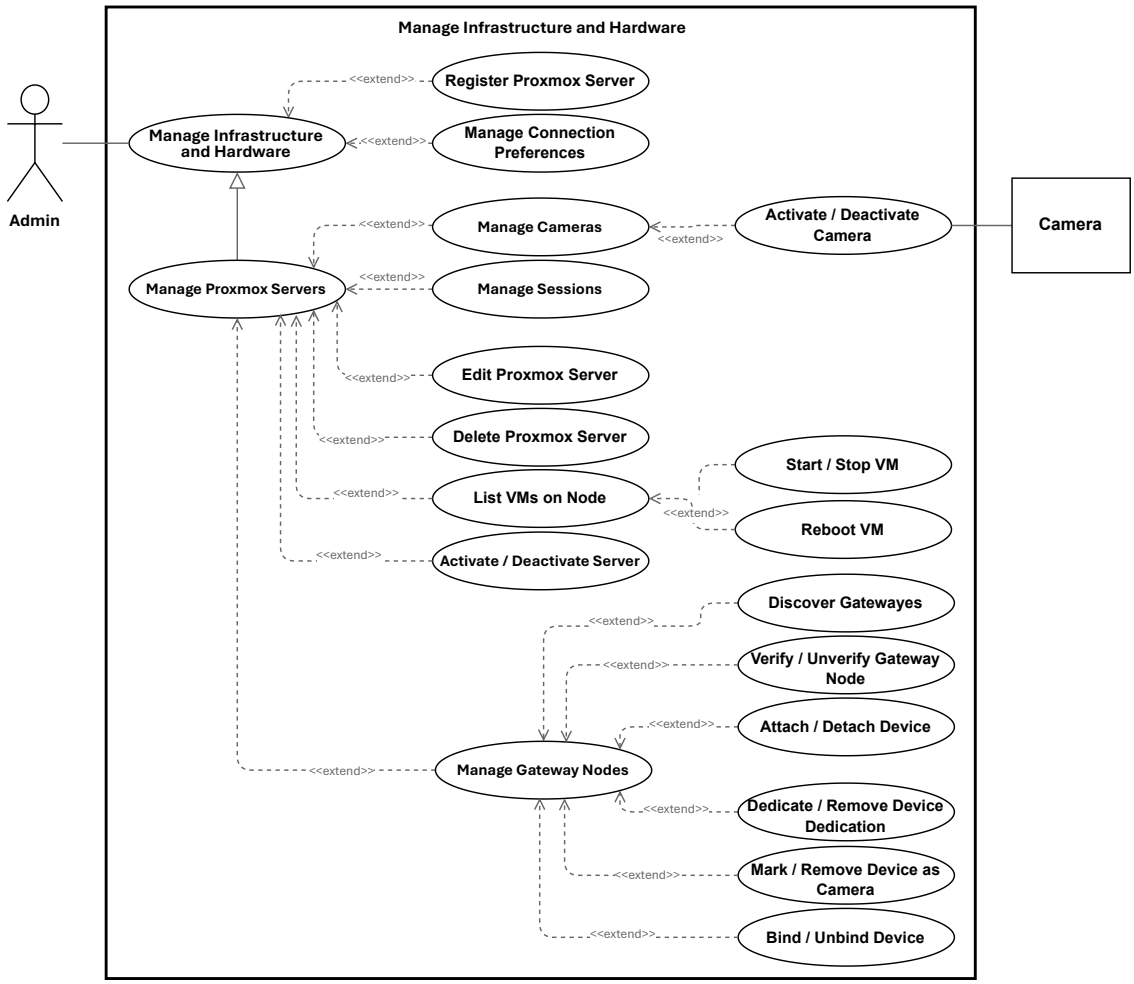

Figure 1.4: Use Case Manage Infrastructure and Hardware

2.2.2.1 Specification of the Use Case Register Proxmox Server

Table 1.8: Use Case Specification: Register Proxmox Server

Title	Register Proxmox Server
Summary	Administrator registers a new Proxmox VE server to enable VM provisioning. System validates connectivity and discovers nodes before adding server to inventory.
Actor	Administrator
Precondition	Administrator is authenticated and authorized. Network access to Proxmox server and valid credentials are available.
Post-condition	Proxmox server is registered and appears in inventory. Discovered nodes are available for provisioning. Configuration is securely stored.
Basic Scenario	1. Administrator opens infrastructure management and selects Register New Server. 2. Administrator enters server details: host address, port, credentials, and optional resource configuration (e.g., VM ID ranges). 3. System validates submitted information. 4. System attempts to connect to Proxmox server using provided credentials. 5. System discovers available cluster nodes. 6. System registers server and stores configuration securely. 7. System displays registered server in inventory with discovered nodes. 8. Administrator can now use this server for provisioning.
Error Scenario	1. If submitted information is invalid or incomplete, system rejects request and displays validation error. 2. If system cannot connect to Proxmox server (unreachable, invalid credentials), system displays error and does not register server. 3. If server is already registered, system informs administrator and suggests updating existing entry. 4. If no cluster nodes are discovered, system flags this and allows administrator to review configuration before proceeding. 5. If system error occurs during registration, system displays error and leaves server unregistered.

2.2.2.2 Specification of the Use Case Edit Proxmox Server

Table 1.9: Use Case Specification: Edit Proxmox Server

Title	Edit Proxmox Server
Summary	Administrator updates the details of an existing Proxmox VE server so the infrastructure inventory remains accurate and usable for provisioning.
Actor	Administrator
Precondition	Administrator is authenticated and authorized. The target Proxmox server exists in the inventory.
Post-condition	The Proxmox server details are updated in the inventory and available for subsequent infrastructure management tasks.
Basic Scenario	1. Administrator opens the server details page and selects the edit option. 2. System displays the current server information. 3. Administrator modifies one or more server details. 4. Administrator submits the changes. 5. System validates the submitted information. 6. System updates the server record. 7. System confirms successful update and shows the revised server details.
Error Scenario	1. Validation failure: system rejects incomplete or invalid input and preserves the entered values for correction. 2. Unauthorized access: system denies the request if the Administrator is not permitted to edit the server. 3. Server not found: system informs the Administrator that the selected server is unavailable. 4. Update failure: system informs the Administrator that the changes could not be saved and keeps the original data unchanged.

2.2.2.3 Specification of the Use Case Delete Proxmox Server

Table 1.10: Use Case Specification: Delete Proxmox Server

Title	Delete Proxmox Server
Summary	Administrator removes a registered Proxmox VE server from the infrastructure inventory when it is no longer needed.
Actor	Administrator
Precondition	Administrator is authenticated and authorized to manage infrastructure. The target Proxmox server exists in the inventory.
Post-condition	The Proxmox server is removed from the inventory and is no longer available for provisioning.
Basic Scenario	1. Administrator selects a Proxmox server and chooses Delete. 2. System asks the Administrator to confirm the deletion. 3. Administrator confirms the deletion. 4. System checks that the server can be deleted. 5. System removes the server from the inventory. 6. System confirms successful deletion.
Error Scenario	1. Deletion canceled: Administrator closes the dialog or selects Cancel; no action is taken. 2. Server not eligible for deletion: system informs the Administrator that the server cannot be removed. 3. Server not found: system informs the Administrator that the selected server is unavailable. 4. Unauthorized access: system denies the request if the Administrator is not permitted to delete infrastructure entries. 5. Deletion failure: system informs the Administrator that the server could not be deleted and keeps the current state unchanged.

2.2.2.4 Specification of the Use Case Manage Connection Preferences

Table 1.11: Use Case Specification: Manage Connection Preferences

Title	Manage Connection Preferences
Summary	Administrator creates, updates, tests, or deletes connection preferences (endpoints, ports, protocols, and credentials) for external platform services (Proxmox, Guacamole, MQTT, Gateway). Preferences are stored securely, auditable, and optionally tested for connectivity.
Actor	Administrator
Precondition	Administrator authenticated and authorized; SecretStore available; UI or API accessible; network access to target service if testing is requested.
Post-condition	Preference persisted with secrets stored securely (encrypted); audit record created; test result recorded if requested; dependent processes (e.g., node discovery) triggered when applicable.
Basic Scenario	1. Administrator opens the Connection Preferences editor and submits a create or update request. 2. Controller validates input and forwards to Service. 3. Service encrypts credentials via SecretStore and persists metadata to the repository. 4. If requested, Service invokes an ExternalClient to test connectivity and records the result. 5. Service writes an audit entry (redacting secrets) and returns the persisted preference and test outcome. 6. Controller responds to the Administrator with success and updated resource.
Error Scenario	1. Validation failure: Controller returns HTTP 422; no changes persisted. 2. Unauthorized: Controller returns HTTP 403; attempt logged. 3. SecretStore/repository failure: Service rolls back and returns HTTP 500. 4. Connection test failure: Service records failed test and returns details; persistence may be allowed or rejected per policy.

2.2.2.5 Specification of the Use Case Discover Gateway Nodes

Table 1.12: Use Case Specification: Discover Gateway Nodes

Title	Discover Gateway Nodes
Summary	Administrator initiates a discovery scan to find all available gateway nodes on the registered Proxmox infrastructure and checks their online status.
Actor	Administrator
Precondition	Administrator is authenticated and authorized to manage infrastructure. At least one Proxmox server is registered and reachable.
Post-condition	System identifies all available gateway nodes in the infrastructure. Each discovered node's online/offline status is verified. A list of discovered nodes with their status is displayed to the administrator.
Basic Scenario	1. Administrator opens the infrastructure management interface. 2. Administrator initiates the *Discover Gateway Nodes* operation. 3. System queries all registered Proxmox servers for available gateway nodes. 4. System checks the online status of each discovered node. 5. System displays a list of discovered nodes, including their names, IP addresses, and current status (online/offline). 6. Administrator can review the list and register newly discovered nodes or update existing entries.
Error Scenario	1. If the administrator lacks permission, the request is denied and an authorization error is displayed. 2. If no Proxmox servers are registered, the system informs the administrator and does not attempt discovery. 3. If a Proxmox server is unreachable during discovery, the system skips it and continues with remaining servers. 4. If no gateway nodes are found, the system displays an empty result and suggests verifying the infrastructure setup. 5. If the discovery operation fails due to a system error, the system displays an error message with any partial results found before the failure.

2.2.2.6 Specification of the Use Case Verify / Unverify Gateway Node

Table 1.13: Use Case Specification: Verify / Unverify Gateway Node

Title	Verify / Unverify Gateway Node
Summary	Allows an administrator to mark a gateway node as trusted (verified) or revoke its trusted status (unverified). The system records verification information and may perform a connectivity test before completing the operation.
Actor	Administrator
Precondition	The administrator is authenticated and authorized. The target gateway node exists in the system.
Post-condition	The verification status of the gateway node is successfully updated and an audit record is created.
Basic Scenario	1. The administrator accesses the gateway inventory. 2. The administrator selects a gateway node. 3. The administrator selects the Verify or Unverify option. 4. The system validates the request. 5. The system performs the connectivity test if requested. 6. The system updates the verification status and stores the verification information. 7. The system records the operation in the audit log. 8. The system displays the updated gateway node information.
Error Scenario	1. **Unauthorized Access:** The system rejects the request and informs the administrator. 2. **Validation Error:** The submitted information is invalid and the system requests correction. 3. **Connectivity Test Failure:** The connectivity test fails and the system notifies the administrator. 4. **Gateway Node Not Found:** The selected gateway node does not exist. 5. **System Error:** An unexpected error occurs and the operation is cancelled.

2.2.2.7 Specification of the Use Case Dedicate / Remove Device Dedication

Table 1.14: Use Case Specification: Dedicate / Remove Device Dedication

Title	Dedicate / Remove Device Dedication
Summary	Administrator permanently assigns a USB device to a specific VM for automatic attachment when sessions are created on that VM, or revokes the assignment.
Actor	Administrator
Precondition	Administrator is authenticated and authorized to manage hardware. The USB device exists in the inventory. The target Proxmox server and VM are valid and reachable.
Post-condition	1. (Dedicate) The device is bound to the target VM. Any current attachment or gateway binding is cleared. The device is ready for automatic attachment to future sessions on that VM. Any linked camera is assigned to the VM. 2. (Remove Dedication) The device is no longer bound to any specific VM. Any linked camera assignment is cleared.
Basic Scenario	1. Administrator navigates to the device details in the admin interface. 2. Administrator selects either *Dedicate to VM* or *Remove Dedication*. 3. For dedication: Administrator specifies the target Proxmox server and VM. 4. System validates the request and checks that the device is not actively in use. 5. System applies the dedication (or removes it) and confirms success. 6. System displays the updated device status and binding information.
Error Scenario	1. If the administrator lacks permission, the request is rejected and an authorization error is displayed. 2. If the specified VM or server does not exist, the system rejects the request and asks for valid input. 3. If the device is currently attached to an active session, the system prevents the operation and instructs the user to detach it first. 4. If the operation fails (e.g., due to unavailable services), the system displays an error and leaves the device in its previous state.

2.2.2.8 Specification of the Use Case Activate / Deactivate Camera

Table 1.15: Use Case Specification: Activate / Deactivate Camera

Title	Activate / Deactivate Camera
Summary	Administrator or engineer in an active session activates a robot camera to enable live streaming, or deactivates it to stop the stream.
Actor	Administrator, Engineer (in active session)
Precondition	Actor is authenticated and authorized to control cameras. Target camera exists and is connected to an online robot. For activation, the camera must not already be streaming.
Post-condition	If activated: Camera begins streaming; live video feed is available to authorized users. If deactivated: Camera stops streaming; video feed is no longer available. The action is recorded for audit.
Basic Scenario	1. Actor opens the camera control interface. 2. Actor selects a camera from the available list. 3. Actor chooses either *Start Stream* or *Stop Stream*. 4. System validates the request and checks that the camera and robot are reachable. 5. System sends the command to the robot and waits for confirmation. 6. Robot acknowledges the request and toggles the camera. 7. System verifies the stream is active or stopped as expected. 8. System displays the updated camera status to the actor. 9. If streaming was activated, the live feed is now available in the interface.
Error Scenario	1. If the actor lacks permission to control cameras, the request is denied and an authorization error is displayed. 2. If the camera or robot is not reachable, the system notifies the actor and does not attempt the operation. 3. If the robot does not respond to the command within a reasonable timeout, the system reports the failure and suggests checking robot connectivity. 4. If the camera is already in the requested state (e.g., already streaming when activation is requested), the system displays a notice. 5. If the stream cannot be established after the robot confirms, the system displays an error and reverts the robot camera state if possible.

2.2.2.9 Specification of the Use Case Manage Sessions

Table 1.16: Use Case Specification: Manage Sessions

Title	Manage Sessions
Summary	Administrator manages VM sessions: view, extend, or terminate. Actions are auditable and may trigger cleanup (Guacamole, Proxmox, hardware).
Actor	Administrator (primary). System services: VMSessionController, VMSessionCleanupService, ExtendSessionService, TerminateVMJob, GuacamoleClient, ProxmoxClient, GatewayService, CameraService (secondary).
Precondition	Administrator is authenticated and authorised. Target session exists and is manageable. Guacamole/Proxmox services are reachable.
Post-condition	On success: session expiry is updated (extend) or session is ended and resources are released (terminate). Guacamole connections are removed and devices/camera controls are released. Actions are audited.
Trigger	Administrator opens session management UI or selects Extend/Terminate on a session.
Basic Scenario	1. Administrator opens session view. 2. System expires overdue sessions and lists active sessions. 3. Administrator selects a session and views details. 4. Administrator chooses Extend or Terminate. 5. System validates authorization and session state. 6. If extend: system applies rules, updates `expires_at`, and returns updated session. 7. If terminate: system deletes Guacamole connection, releases camera controls and USB attachments, optionally stops or reverts the VM, and marks session as ended. 8. System refreshes the list and displays confirmation.
Error Scenario	1. If unauthorized, system returns HTTP 403 and logs the attempt. 2. If the session is already ended, system returns no-op or validation error. 3. If extension violates quota or time-window rules, system returns validation error and does not update expiry. 4. If termination cleanup fails (Guacamole/Proxmox/Hardware), system logs the error, retries according to job policy, and returns a warning; the session is force-marked as ended for later reconciliation when safe.

2.2.3 Refinement of the Use Case Manage Content Studio

Figure 1.5 presents the refinement of the Use Case Manage Content Studio for the Teacher actor. This use case includes the following sub-use cases:

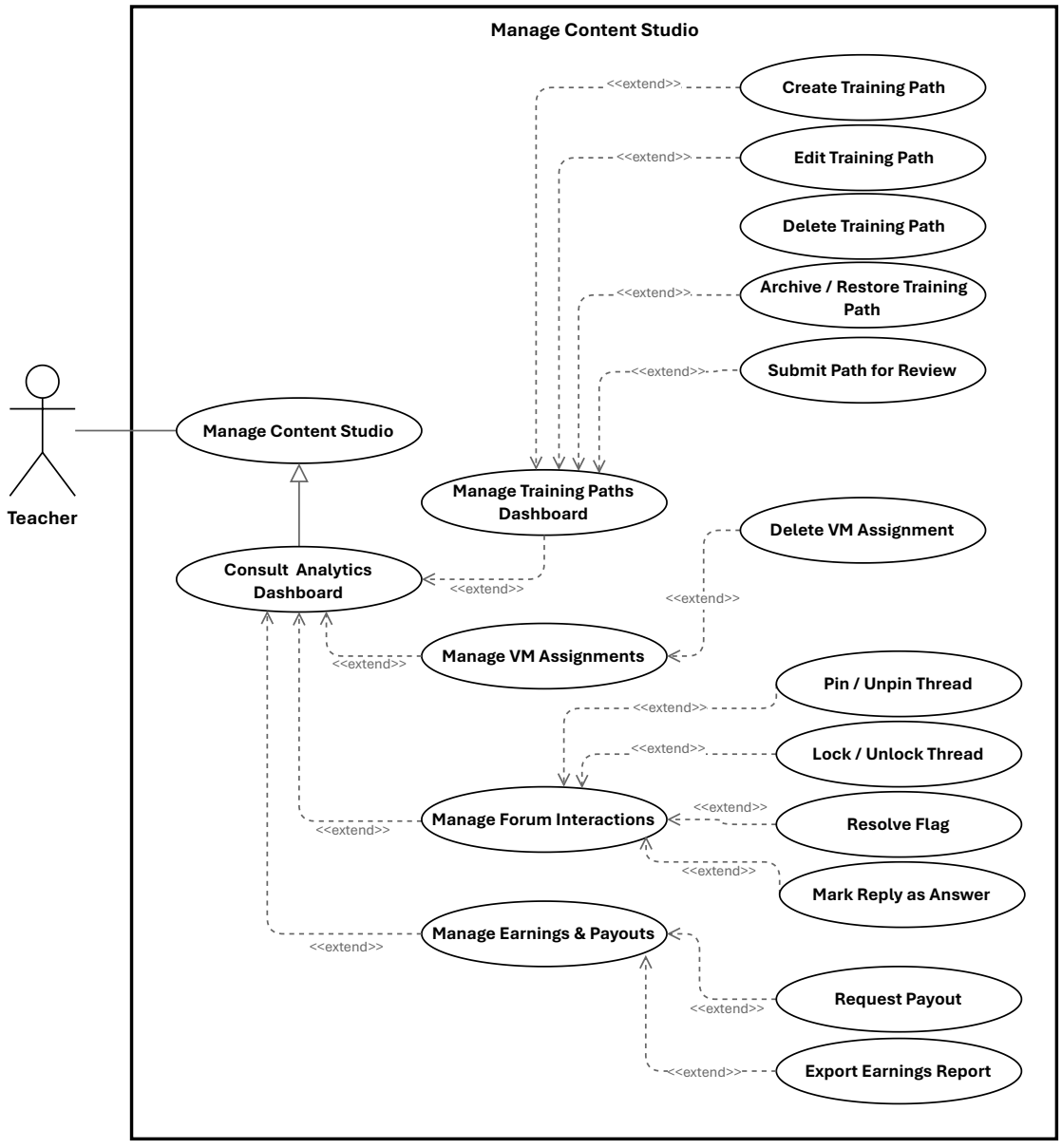

Figure 1.5: Use Case Manage Content Studio

2.2.3.1 Specification of the Use Case Create Training Path

2.2.3.2 Specification of the Use Case Edit Training Path

Table 1.17: Use Case Specification: Edit Training Path

Title	Edit Training Path
Summary	Teacher updates the details of an existing training path so its content, description, and related information remain accurate and aligned with the teaching plan.
Actor	Teacher
Precondition	Teacher is authenticated, owns the target training path, and the path is available for editing.
Post-condition	The training path details are updated and the Teacher can view the revised information. On failure, the original information remains unchanged.
Basic Scenario	1. Teacher opens the training path editor. 2. System displays the current training path information. 3. Teacher modifies one or more editable fields. 4. Teacher submits the changes. 5. System validates the submitted information. 6. System updates the training path. 7. System confirms successful update and shows the revised details.
Error Scenario	1. Validation failure: system rejects incomplete or invalid input and preserves the entered values for correction. 2. Unauthorized access: system denies the request if the Teacher is not allowed to edit the training path. 3. Training path not found: system informs the Teacher that the requested path is unavailable. 4. Update failure: system informs the Teacher that the changes could not be saved and keeps the original data unchanged.

2.2.3.3 Specification of the Use Case Delete Training Path

Table 1.18: Use Case Specification: Delete Training Path

Title	Delete Training Path
Summary	Teacher permanently removes one of their own training paths after confirming the action.
Actor	Teacher
Precondition	Teacher is authenticated, owns the target training path, and the path is eligible for deletion.
Post-condition	The training path is removed and no longer appears in the teacher's active listings.
Basic Scenario	1. Teacher selects a training path and chooses Delete. 2. System asks the Teacher to confirm the deletion. 3. Teacher confirms the deletion. 4. System verifies that the Teacher is allowed to delete the training path. 5. System removes the training path from active listings. 6. System confirms successful deletion.
Error Scenario	1. Deletion canceled: Teacher closes the dialog or selects Cancel; no action is taken. 2. Training path not eligible for deletion: system informs the Teacher that the path cannot be deleted. 3. Training path not found: system informs the Teacher that the requested path is unavailable. 4. Unauthorized access: system denies the request if the Teacher does not own the training path. 5. Deletion failure: system informs the Teacher that the path could not be deleted and keeps the current state unchanged.

2.2.3.4 Specification of the Use Case Archive / Restore Training Path

Table 1.19: Use Case Specification: Archive / Restore Training Path

Title	Archive / Restore Training Path
Summary	Teacher archives a training path to hide it from active listings while preserving content; later restores it to active listings.
Actor	Teacher
Precondition	Teacher is authenticated and owns the training path; it exists in the system.
Post-condition	Archive: path marked archived, removed from active listings, content preserved. Restore: archived flag cleared, path returns to active listings.
Basic Scenario	1. Teacher selects path and chooses "Archive". 2. System asks for confirmation. 3. Teacher confirms. 4. System verifies ownership and archivable state. 5. System marks path as archived, records audit, and removes from active listings. 6. System confirms success; item no longer appears in active lists. 7. Later, Teacher navigates to archived items and chooses "Restore". 8. System asks for confirmation. 9. Teacher confirms. 10. System verifies ownership and archived state, clears archived status, records audit, and restores to active listings. 11. System confirms success; item appears again in active lists.
Error Scenario	1. If Teacher does not own path, system denies action and displays authorization message. 2. If path is not found or already in requested state, system displays validation message. 3. If system persistence error occurs, system preserves consistent state, logs incident, and advises retry. 4. If auxiliary indexing fails, system completes in primary store, records inconsistency for reconciliation, and informs Teacher that listings may update shortly.

2.2.3.5 Specification of the Use Case Delete VM Assignment

Table 1.20: Use Case Specification: Delete VM Assignment

Title	Delete VM Assignment
Summary	Teacher deletes an existing VM assignment they created; the system releases any provisioned VM and remote connections, records the action for audit, and notifies relevant parties.
Actor	Teacher (primary). System (secondary).
Precondition	Teacher is authenticated and authorized to manage the assignment; the VM assignment exists and is eligible for deletion.
Post-condition	On success: the assignment is removed (or soft-deleted per policy); associated external resources are released or scheduled for cleanup; an audit entry is recorded; affected users are notified.
Basic Scenario	1. Teacher locates the VM assignment and chooses Delete. 2. System asks the Teacher to confirm the deletion. 3. Teacher confirms the deletion. 4. System verifies the Teacher's authorization and that the assignment may be deleted. 5. System releases or schedules cleanup of any provisioned VM and removes any remote access/connection resources associated with the assignment. 6. System removes (or marks as deleted) the assignment record and records the deletion in the audit log. 7. System notifies the Teacher and any other relevant parties about the deletion and cleanup status. 8. The UI reflects the deletion and updated resource status.
Error Scenario	1. Assignment not found: system informs the Teacher and makes no changes. 2. Unauthorized (not owner or insufficient privileges): system denies the action and explains required permissions. 3. External resource cleanup fails (e.g., VM removal or remote connection deletion): system logs the failure, retries or enqueues a cleanup job, may perform a soft-delete of the assignment per policy, and notifies the Teacher of the partial outcome and next steps. 4. System error during deletion: system preserves consistent state, records the incident for investigation, and advises the Teacher to retry later or contact support.

2.2.3.6 Specification of the Use Case Pin / Unpin Thread

Table 1.21: Use Case Specification: Pin / Unpin Thread

Title	Pin / Unpin Thread
Summary	Teacher pins or unpins a forum thread within a training path to highlight or remove emphasis. The system updates the thread's visibility and records the moderation action for audit.
Actor	Teacher (primary)
Precondition	Teacher is authenticated and has moderation permission for the training path; the target thread exists and is not deleted.
Post-condition	The thread's pinned state is updated and persisted; forum listings and thread views reflect the change; a moderation audit entry is recorded.
Basic Scenario	1. Teacher opens the forum inbox or thread detail and selects the target thread. 2. Teacher chooses the Pin or Unpin action. 3. System verifies the teacher's authorization and that the thread is in a valid state for moderation. 4. System updates the thread's pinned status and persists the change. 5. System records a moderation audit entry (actor, action, thread identifier, timestamp). 6. System updates forum listings and thread ordering so the change is visible to authorized users. 7. UI displays confirmation and the thread's new pinned state.
Error Scenario	1. Unauthorized: If the teacher lacks moderation permission, the system rejects the action and informs the teacher. 2. Not found / invalid state: If the thread is missing or deleted, the system reports the condition and takes no action. 3. Persistence failure: If the update cannot be saved, the system aborts the change, logs the failure, records an error audit, and notifies the teacher to retry later.

2.2.3.7 Specification of the Use Case Lock / Unlock Thread

Table 1.22: Use Case Specification: Lock / Unlock Thread

Title	Lock / Unlock Thread
Summary	Teacher locks a thread to prevent further replies or unlocks it to allow discussion; the system enforces the state, records a moderation audit entry, and optionally notifies subscribers.
Actor	Teacher (primary)
Precondition	Teacher is authenticated and authorised to moderate the training path containing the thread; the thread exists and is not archived.
Post-condition	On success: the thread's locked state is set (locked/unlocked) and persisted; a moderation audit entry is recorded; subscribers may be notified of the change.
Basic Scenario	1. Teacher selects Lock or Unlock for a target thread and confirms the action. 2. System verifies the teacher's moderation permission and that the thread is in a valid state for the requested action. 3. System updates and persists the thread's locked state. 4. System records a moderation audit entry with actor, action, thread identifier and timestamp. 5. System updates forum listings/permission enforcement so replies are blocked or allowed accordingly. 6. System returns confirmation to the teacher and the UI reflects the updated thread metadata.
Error Scenario	1. Unauthorized: If the teacher lacks moderation permission, the system rejects the action and informs the teacher. 2. Not found / invalid state: If the thread cannot be located or is archived, the system reports the condition and takes no action. 3. Persistence / notification failure: If the locked-state update or notification cannot be saved/sent, the system aborts or rolls back the change, logs the failure, records an error audit, and informs the teacher to retry later.

2.2.3.8 Specification of the Use Case Resolve Flag

Table 1.23: Use Case Specification: Resolve Flag

Title	Resolve Flag
Summary	Teacher reviews a flagged discussion thread within an owned training path, takes an appropriate moderation action, and clears the flag so the thread is no longer treated as requiring attention.
Actor	Teacher
Precondition	Teacher is authenticated and owns the training path containing the flagged thread; the target thread exists and is currently flagged.
Post-condition	The flag is cleared on the target thread and persisted; the thread is removed from flagged-content views; the action is recorded for audit and the teacher receives confirmation.
Basic Scenario	1. Teacher opens the forum inbox or flagged-content view. 2. System displays flagged threads belonging to the teacher's training paths. 3. Teacher selects a flagged thread and reviews the content. 4. Teacher chooses the Resolve Flag action. 5. System verifies ownership and that the thread is still flagged. 6. System clears the flag, persists the updated moderation state, and records an audit entry. 7. System confirms success and the thread is removed from flagged-content views.
Error Scenario	1. Unauthorized: If the teacher does not own the training path, the system rejects the action and keeps the flag unchanged. 2. Invalid state: If the thread is no longer flagged or cannot be found, the system informs the teacher and makes no change. 3. Persistence failure: If the unflag operation cannot be saved, the system preserves the previous state, logs the failure, records an error audit, and prompts the teacher to retry later.

2.2.3.9 Specification of the Use Case Request Payout

Table 1.24: Use Case Specification: Request Payout

Title	Request Payout
Summary	Teacher requests withdrawal of an amount from available earnings. The system validates amount and payment method according to payout policies, creates a pending payout request, and confirms submission.
Actor	Primary: Teacher. Secondary: System.
Precondition	Teacher is authenticated and has earnings data with a non-zero available balance. Earnings/payout interface is available.
Post-condition	Payout request is created with status pending and associated with the teacher account. System confirms successful submission.
Basic Scenario	1. Teacher opens the earnings section and selects Request Payout. 2. System displays payout form with available balance and payment options. 3. Teacher enters payout amount and selects payment method. 4. Teacher submits the request. 5. System validates fields and checks available balance and minimum threshold. 6. System verifies amount and payment method validity. 7. System creates payout request with pending status and confirms submission to the Teacher.
Error Scenario	1. Amount exceeds available balance: system rejects request and displays a validation message; Teacher may retry with a valid amount. 2. Amount below minimum threshold: system rejects request and prompts Teacher to increase amount; Teacher may retry. 3. Invalid payment method: system rejects request and requests a valid method; Teacher may retry. 4. Persistence or service failure: system preserves previous state, logs the incident, records an error audit, and informs the Teacher to retry later.

2.2.3.10 Specification of the Use Case Export Earnings Report

Table 1.25: Use Case Specification: Export Earnings Report

Title	Export Earnings Report
Summary	Teacher exports a CSV report of completed earnings transactions for a selected date range so the income history can be reviewed, reconciled, or archived for accounting purposes. The system generates a downloadable file without changing any financial records.
Actor	Primary: Teacher. Secondary: System.
Precondition	Teacher is authenticated and can access the earnings page. Earnings data is available in the system. If a date range is provided, it is valid.
Post-condition	CSV report is generated for the teacher's completed payments within the selected date range. Browser receives streamed download named earnings-YYYY-MM-DD.csv. No financial data is modified.
Basic Scenario	1. Teacher opens the earnings page and requests an export. 2. System resolves the authenticated teacher and determines the export date range (selected or default). 3. System generates a CSV from the teacher's completed payments for the date range. 4. System streams the CSV file to the teacher's browser and the download starts.
Error Scenario	1. Invalid or empty date range: supplied date range is malformed or inverted; system rejects the request and displays a validation message; no download occurs. 2. Export processing failure: data retrieval or CSV generation fails; system aborts the export, preserves current state, logs the incident, and surfaces a recoverable error. 3. No matching payments: selected date range contains no completed payments; system generates a CSV with header row only and streams it successfully.

2.2.4 Refinement of the Use Case Manage Enrolled Training Paths

Figure 1.6 presents the refinement of the Use Case Manage Enrolled Training Paths for the Engineer actor. This use case includes the following sub-use cases:

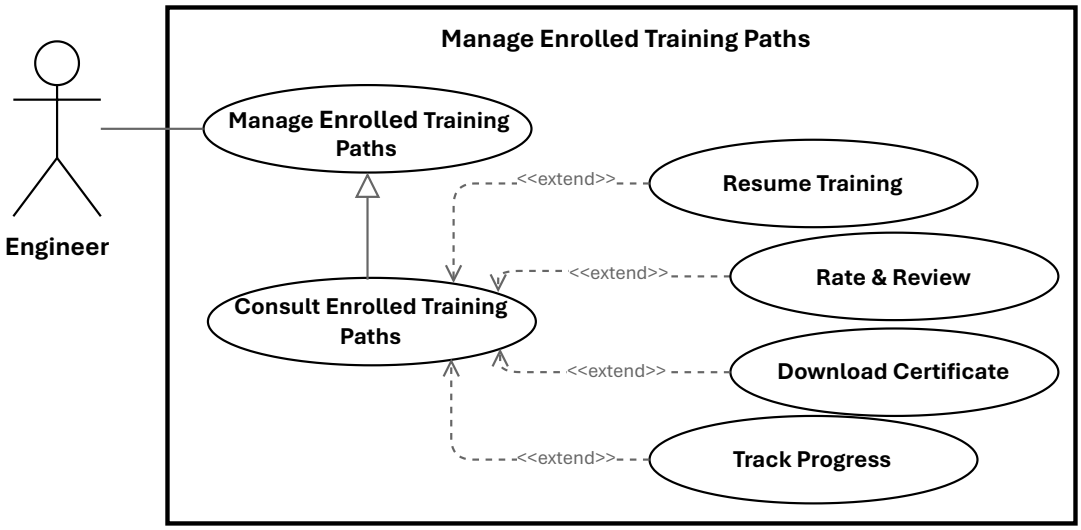

Figure 1.6: Use Case Manage Enrolled Training Paths

2.2.4.1 Specification of the Use Case Resume Training

Table 1.26: Use Case Specification: Resume Training

Title	Resume Training
Summary	Learner resumes a previously started training unit or module at the last saved position so they can continue without losing progress.
Actor	Primary: Engineer (Learner). Secondary: System.
Precondition	Learner is authenticated and enrolled in the training path; the platform stores progress for the unit.
Post-condition	On success: learner resumes at the saved position and the system continues tracking and persisting progress.
Basic Scenario	1. Learner clicks "Resume" on the training unit page. 2. System verifies the learner's authentication status. 3. System verifies that the learner is enrolled and has access to the training unit. 4. System retrieves the last saved progress for the selected unit. 5. System loads the content metadata and associated learning resource. 6. System returns the resume details, including the content location and the starting position. 7. Learner starts playback or continues reading from the saved position. 8. System continues to record progress updates as the learner resumes training.
Error Scenario	1. Authentication failure: session is invalid or expired; the system blocks access and requests re-authentication. 2. Content retrieval failure: the learning resource cannot be loaded; the system displays an error and may allow retry. 3. Progress persistence failure: the system cannot save updates; the previous state is preserved and a recoverable error is shown.

2.2.4.2 Specification of the Use Case Rate & Review

Table 1.27: Use Case Specification: Rate & Review

Title	Rate & Review
Summary	Engineer submits a star rating and optional written review for a training path. The system validates the submission, stores the review, recalculates the training path's average rating, and applies moderation rules when required.
Actor	Engineer
Precondition	Engineer is authenticated and has access to the training path review interface; the target training path exists.
Post-condition	On success: the review is saved or updated, the training path average rating is recalculated, and the appropriate visibility or approval state is set.
Post-condition (Failure)	No review is stored or updated; the previous state is preserved and an error is reported.
Basic Scenario	1. Engineer opens the review form for a training path. 2. Engineer selects a star rating and optionally enters review text, then submits the form. 3. System validates the engineer's eligibility and the submitted input. 4. System saves the review and recalculates the training path's average rating. 5. If an existing review is detected, the system updates the existing record. 6. System confirms the submission, or marks it as pending if moderation is required.
Error Scenario	1. Missing rating: the system rejects the submission with a validation error. 2. Persistence failure: the system aborts the save, preserves the previous state, logs the error, and notifies the engineer. 3. Moderation required: the system stores the review as pending and hides it until approval.

2.2.4.3 Specification of the Use Case Download Certificate

Table 1.28: Use Case Specification: Download Certificate

Title	Download Certificate
Summary	Engineer requests and downloads a completion certificate (PDF) for a training path they have completed. The system verifies eligibility, retrieves or generates the certificate, and returns it as a downloadable file while logging the action.
Actor	Primary: Engineer. Secondary: System.
Precondition	The engineer is authenticated; the engineer has completed the training path and a certificate record exists or can be generated.
Post-condition	The engineer receives the certificate file; a download audit record is created; certificate delivery metadata is recorded.
Basic Scenario	1. Engineer requests a certificate download. 2. System authenticates the request and identifies the engineer. 3. System verifies that the engineer is eligible to download the certificate. 4. System checks whether a certificate file already exists. 5. If a file exists, the system retrieves it; otherwise, the system generates the certificate and stores it. 6. System returns the certificate file or a download link to the engineer. 7. The download starts in the browser or client application. 8. System records the successful download in the audit log.
Error Scenario	1. Unauthenticated request: the system denies access and requests re-authentication. 2. Not eligible: the system rejects the request if the engineer is not entitled to the certificate. 3. Certificate generation or storage error: the system aborts the operation, preserves the current state, and logs the failure.

2.2.5 Refinement of the Use Case Manage Virtual Lab

Figure 1.7 presents the refinement of the Use Case Manage Virtual Lab for the Engineer actor. This use case includes the following sub-use cases:

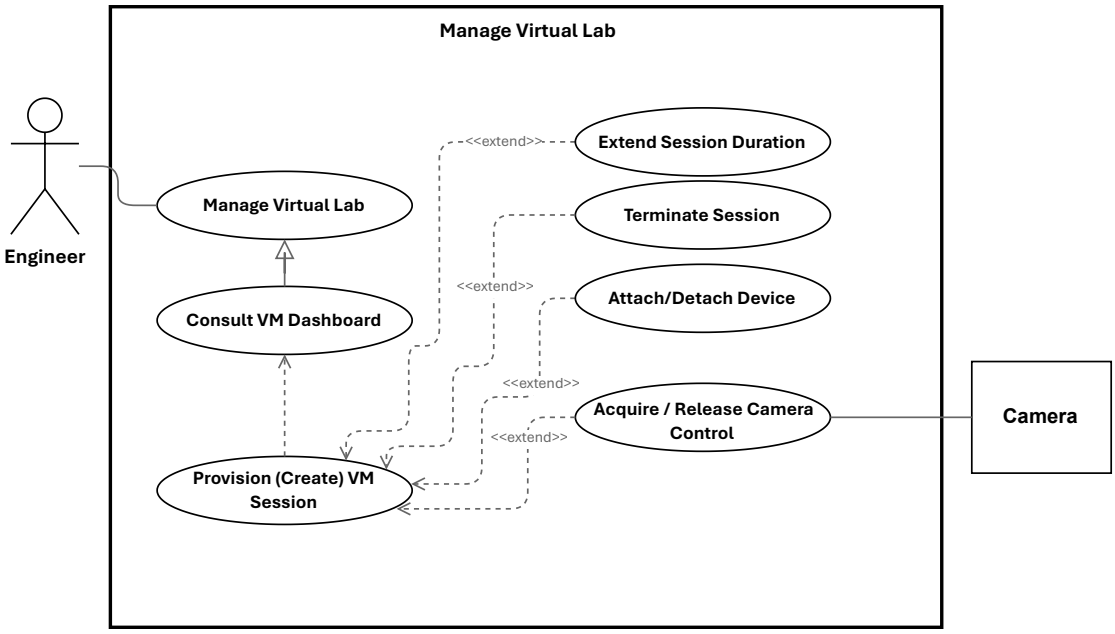

Figure 1.7: Use Case Manage Virtual Lab

2.2.5.1 Specification of the Use Case Provision (Create) VM Session

Table 1.29: Use Case Specification: Provision (Create) VM Session

Title	Provision (Create) VM Session
Summary	Engineer requests an ephemeral VM session by selecting a template and duration. The system validates the request, checks quota, provisions a virtual machine, creates remote access, persists the session, and schedules cleanup.
Actor	Engineer
Precondition	Engineer is authenticated and authorized; the requested template exists and is available; the virtualization and remote-access services are reachable; background processing is operational.
Post-condition	A VM session record exists with status active or provisioning; the VM is provisioned with a host and IP address; remote-access details are created and associated; a cleanup job is scheduled.
Basic Scenario	1. Engineer submits a VM session request with a template and duration. 2. System validates the request and authorizes the engineer. 3. System checks whether the engineer can create a new session. 4. System selects an available node for provisioning. 5. System provisions the virtual machine from the selected template. 6. System waits until provisioning completes and retrieves the VM details. 7. System creates and stores the VM session record. 8. System creates the remote-access connection and associates it with the session. 9. System updates the session status to active and schedules cleanup after the requested duration. 10. System returns the created VM session to the engineer.
Error Scenario	1. Quota exceeded: the system rejects the request and displays an explanatory message. 2. Provisioning failure: the system aborts the operation, attempts cleanup, and marks the session as failed. 3. Persistence failure: the system attempts to roll back the created VM and reports an error. 4. Remote-access setup failure: the system records a partial failure and either schedules remediation or rolls back according to policy.

2.2.5.2 Specification of the Use Case Extend Session Duration

Table 1.30: Use Case Specification: Extend Session Duration

Title	Extend Session Duration
Summary	Engineer requests additional time for an active VM session. The system validates eligibility, checks quota or billing constraints, updates the session expiry, reschedules cleanup, and notifies the engineer.
Actor	Engineer. Secondary: System services.
Precondition	Engineer is authenticated and owns an active VM session; session information is available and the quota or billing service can be consulted.
Post-condition	On success: the session expiry is updated and saved; the cleanup job is rescheduled; quota or billing counters are updated; the engineer is notified and an audit entry is created.
Post-condition (Failure)	No expiry change is made; quota or billing state remains unchanged; the engineer receives an explanatory error and the incident is logged.
Basic Scenario	1. Engineer requests an extension for an active VM session. 2. System authenticates the request and verifies the engineer's ownership of the session. 3. System checks that the session is active and eligible for extension. 4. System verifies quota or billing authorization. 5. If the request is valid, system updates the session expiry, reschedules cleanup, records an audit entry, and notifies the engineer. 6. System returns a success response with the updated session information.
Error Scenario	1. Session not found or not owned: the system rejects the request and returns an authorization or not-found error. 2. Session expired or terminated: the system rejects the request and indicates that a new session must be created. 3. Quota or billing failure: the system rejects the extension request. 4. Persistence or scheduler failure: the system rolls back the update if possible, logs the error, and raises an operational alert.

2.2.5.3 Specification of the Use Case Terminate Session

Table 1.31: Use Case Specification: Terminate Session

Title	Terminate Session
Summary	Engineer ends an active VM session. The system validates authorization, revokes remote access, detaches devices, stops or schedules VM deletion, updates session state, logs the action, and notifies stakeholders.
Actor	Engineer (session owner). Secondary: System.
Precondition	Engineer is authenticated and authorized; the session exists and is in an active state.
Post-condition	On success: the session is marked terminated, remote access is revoked, attached devices are released, the VM is deleted or scheduled for cleanup, an audit entry is recorded, and notifications are sent.
Basic Scenario	1. Engineer issues a terminate request for the session through the UI or API. 2. System authenticates the request and verifies authorization. 3. System marks the session as terminating. 4. System revokes remote access, detaches devices, deletes the VM or schedules cleanup, cancels scheduled jobs, records an audit entry, and notifies stakeholders. 5. System returns a success response with the updated session state.
Error Scenario	1. Authorization failure: the system returns 403 and no change is applied. 2. Persistence failure: the system aborts the operation, logs the error, and returns 500. 3. External API failure: Proxmox or Guacamole errors are retried per policy; persistent failures mark the session with cleanup errors and notify operators.

2.2.5.4 Specification of the Use Case Acquire / Release Camera Control

Table 1.32: Use Case Specification: Acquire / Release Camera Control

Title	Acquire / Release Camera Control
Summary	Engineer acquires exclusive PTZ control of a laboratory camera and later releases it so other users may use the device. The system manages leases, queues, and notifications to coordinate shared access.
Actor	Primary: Engineer. Secondary: Other engineers/users, System Administrator (override), NotificationService, CameraDevice.
Precondition	Engineer is authenticated, has an active session, and the camera is online with PTZ capabilities; control-state tracking is available.
Post-condition	On success: ownership is assigned with a lease expiry; PTZ commands are relayed and acknowledged; on release or lease expiry the camera becomes available and subscribers are notified.
Basic Scenario	1. Engineer requests control of camera C. 2. System verifies authentication, session, and device capabilities. 3. If available, system reserves control and issues a lease with an expiry. 4. System returns success; UI enables PTZ controls and broadcasts a ControlGranted notification. 5. While holding control, the engineer sends PTZ commands which the system relays to the device and acknowledges. 6. Engineer releases control or lease expires; system clears ownership, marks the camera available, and broadcasts ControlReleased.
Error Scenario	1. Camera offline or unreachable: request fails and no ownership change occurs. 2. Unsupported capability: acquire is rejected and PTZ controls remain disabled. 3. Persistence/write failure: system attempts rollback, logs the error, and alerts operations; ownership remains unchanged. 4. Camera busy: system returns BUSY; UI may offer a queue option and the user can re-enter the flow when promoted.

3 Class Diagram

Figure 1.8 presents the general class diagram of our application.

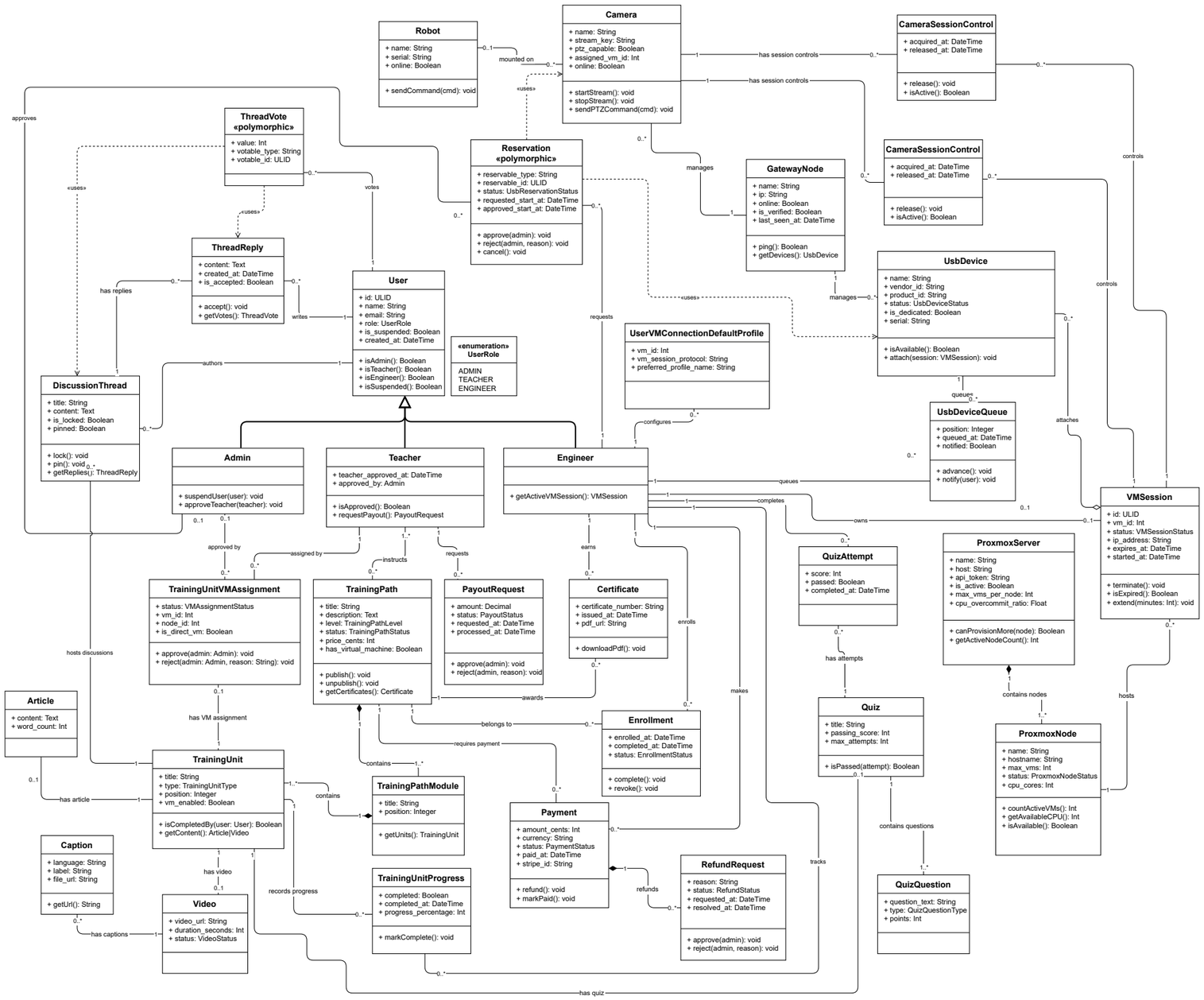

Figure 1.8: Class Diagram of the IoT-REAP Platform

Class User: The class User represents a general platform user and serves as the base entity for all person-like actors. It contains identifying and contact information (such as ULID, name and email), authentication attributes, role membership and suspension flags, and timestamps. User encapsulates shared behaviour and relationships that are common to specialized user types.

Class Admin: The class Admin represents platform administrators with elevated privileges. Admins are responsible for system governance tasks such as user suspension, quota management, teacher approvals, and oversight of infrastructure and financial operations.

Class Teacher: The class Teacher models instructional staff who author and manage training content. Teachers have approval metadata, authoring capabilities, and interfaces for requesting payouts and reviewing learner submissions.

Class Engineer: The class Engineer represents practitioners who consume training materials and operate remote laboratory resources. Engineers interact with VM sessions, hardware reservations, and robot control features.

Class UserRole: An enumeration that defines the possible user roles (ADMIN, TEACHER, ENGINEER). UserRole is used to gate permissions and to differentiate behavior and available features at runtime.

Class TrainingPath: The class TrainingPath models a purchasable or enrollable course consisting of a sequence of modules and units. It stores metadata such as title, description, level, pricing and publication status.

Class TrainingPathModule: TrainingPathModule groups related TrainingUnit objects into a coherent module or chapter within a TrainingPath. Modules provide ordering, summarization and module-level metadata.

Class TrainingUnit: The class TrainingUnit represents an atomic learning unit (for example a lesson or exercise). A unit may be of various types (Article, Video, Quiz) and tracks progress, assignment status and completion criteria.

Class Article: Article is a content subtype representing textual learning material, including body content, attachments and metadata for rendering.

Class Video: Video is a content subtype representing embedded or hosted video media used within a TrainingUnit.

Class Quiz: The Quiz class models assessment units composed of questions. It references QuizQuestion objects and aggregates attempts and scoring rules.

Class QuizQuestion: QuizQuestion represents individual questions within a Quiz, including question text, possible answers and correct answer metadata.

Class QuizAttempt: QuizAttempt records a learner's attempt at a Quiz, including answers chosen, score achieved and timestamps.

Class DiscussionThread: DiscussionThread models a forum-style discussion attached to a TrainingUnit. It aggregates posts and provides moderation and thread metadata.

Class ThreadReply: ThreadReply represents an individual reply within a DiscussionThread; replies may be accepted, voted on, and linked to their authors.

Class Enrollment: Enrollment links a User to a TrainingPath, capturing enrolment date, progress, and entitlement to access path materials and resources.

Class TrainingUnitProgress: TrainingUnitProgress tracks a learner's status within a specific TrainingUnit (e.g., not started, in progress, completed) and records timestamps and completion evidence.

Class Certificate: Certificate models credential issuance after successful completion of a TrainingPath or assessment; it contains recipient data, issuance date and validation metadata.

Class VMSession: VMSession represents a provisioned virtual machine session used by Engineers. It captures VM identifiers, node allocation, status, expiry time and associations to user reservations, Guacamole connections and cleanup jobs.

Class TrainingUnitVMAssignment: This class links TrainingUnits to VM templates or assignments that require a live virtual machine; it stores assignment status, VM identifiers and approval metadata.

Class UserVMConnectionDefaultProfile: UserVMConnectionDefaultProfile captures per-user connection preferences and default profiles for remote desktop or terminal connections to provisioned VMs.

Class ProxmoxServer: ProxmoxServer represents the Proxmox deployment as a logical entity, containing configuration references, API endpoints and a set of ProxmoxNode objects.

Class ProxmoxNode: ProxmoxNode models an individual compute node within the Proxmox cluster; nodes host VM instances and expose resource metrics used by the scheduler and load balancer.

Class GatewayNode: GatewayNode models infrastructure gateways (for example a dedicated VM or host that proxies hardware or network access) and coordinates services such as USB/IP and camera streaming.

Class UsbDevice: UsbDevice models physical USB devices available for reservation and attachment to VMs, tracking device identifiers, availability status and reservation links.

Class Camera: Camera represents camera hardware in the inventory, including streaming endpoints, formats supported and reservation/attachment state.

Class CameraSessionControl: CameraSessionControl encapsulates session-level control for cameras (attach/detach, format negotiation and stream lifecycle) when cameras are assigned to a VM session.

Class Reservation: Reservation is a polymorphic record representing a requested allocation of a reservable resource (USB device, camera, VM slot). It stores requested and approved time windows, status and references to the reservable entity.

Class UsbDeviceQueue: UsbDeviceQueue models the waiting queue for an unavailable USB device, enabling FIFO assignment and notifications when a device becomes available.

Class Robot: Robot models a remote or simulated robot endpoint that can be reserved or controlled from a VM session; it includes identifiers, command interfaces and telemetry endpoints.

Class Payment: Payment records financial transactions (amount, currency, status, user and training path references) and serves as the source for refund and payout workflows.

Class RefundRequest: RefundRequest models a user-initiated refund claim, including reason, linked payment, eligibility checks and administrative review status.

Class PayoutRequest: PayoutRequest represents teacher-initiated requests to withdraw earned funds; it includes payout amount, recipient details and approval state. withdraw earned funds; it includes payout amount, recipient details and approval state.

4 Sequence Diagrams

Sequence diagrams illustrate the temporal interaction and message flow between system actors and components, showing how workflows unfold over time. Activity diagrams model decision points, parallel flows, and state transitions, emphasizing branching conditions and process paths where multiple outcomes are possible. Together, these diagram types capture both the choreography and decision logic of major platform workflows.

4.1 Manage Session Sequence

Figure 1.9 illustrates the manage session sequence.

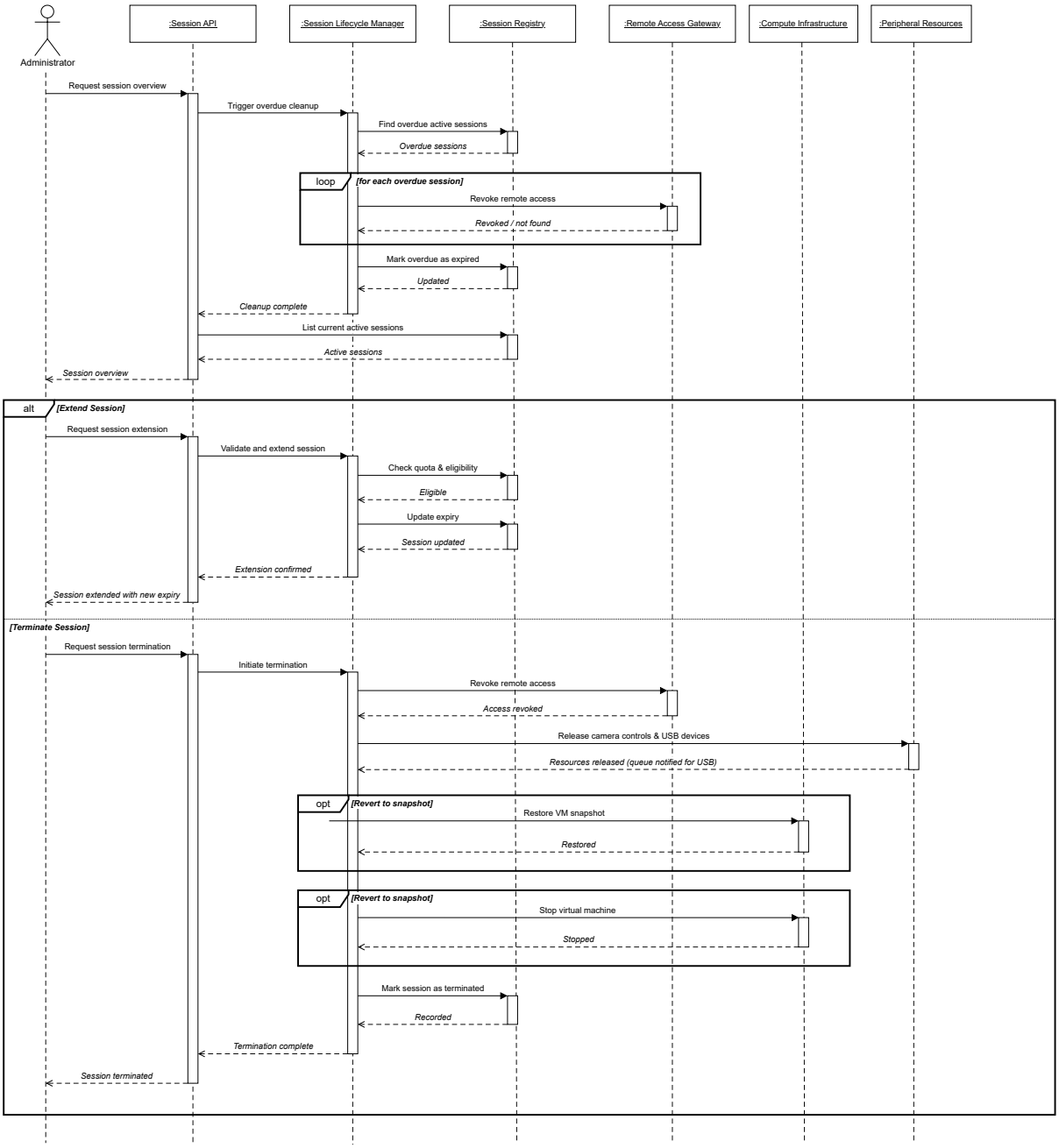

Figure 1.9: Sequence Diagram: Manage Session

4.2 Discover Gateway Node Sequence

Figure 1.10 illustrates the discover gateway node sequence.

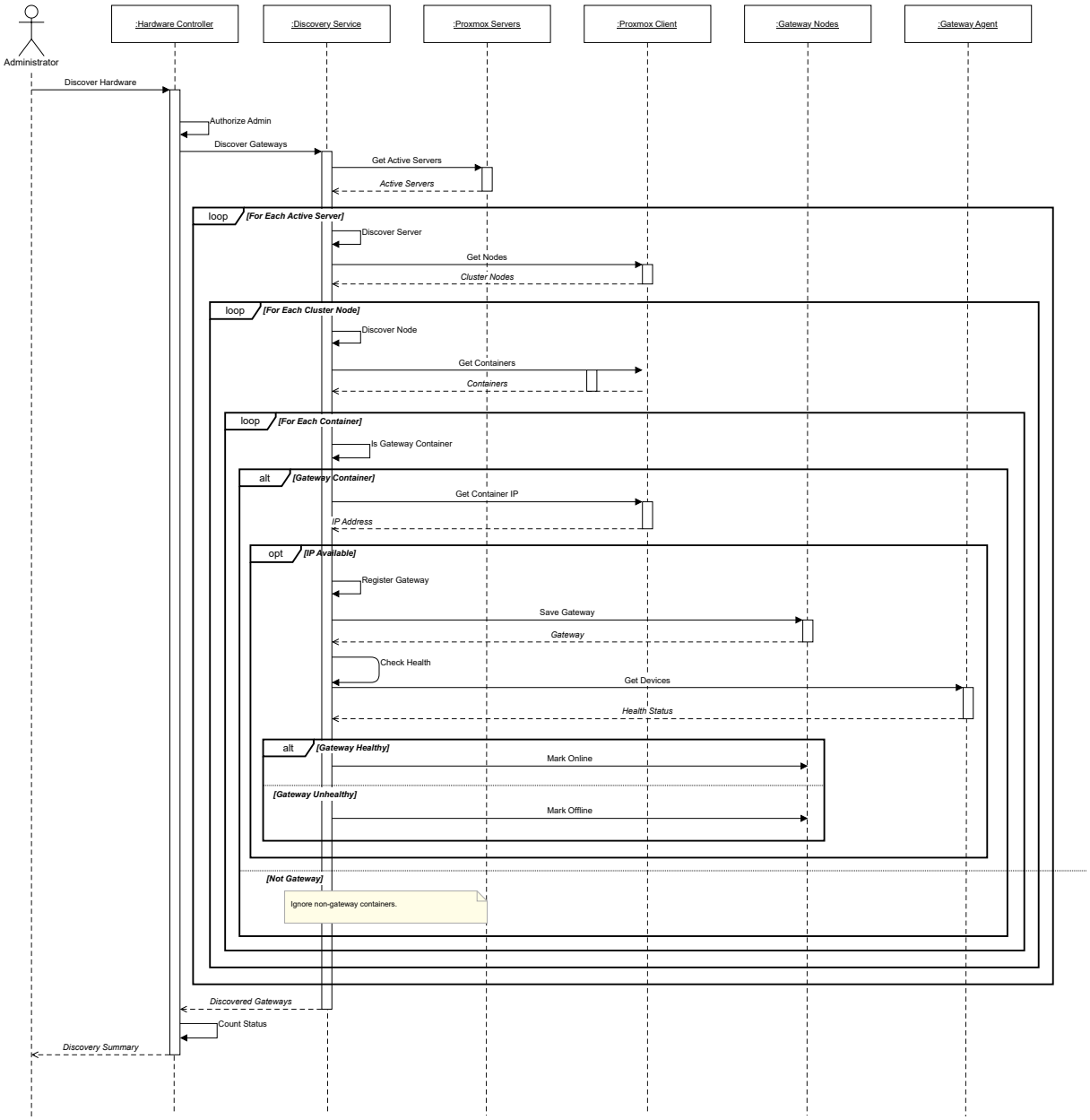

Figure 1.10: Sequence Diagram: Discover Gateway Node

4.3 Payout Request Sequence

Figure 1.11 illustrates the payout request sequence.

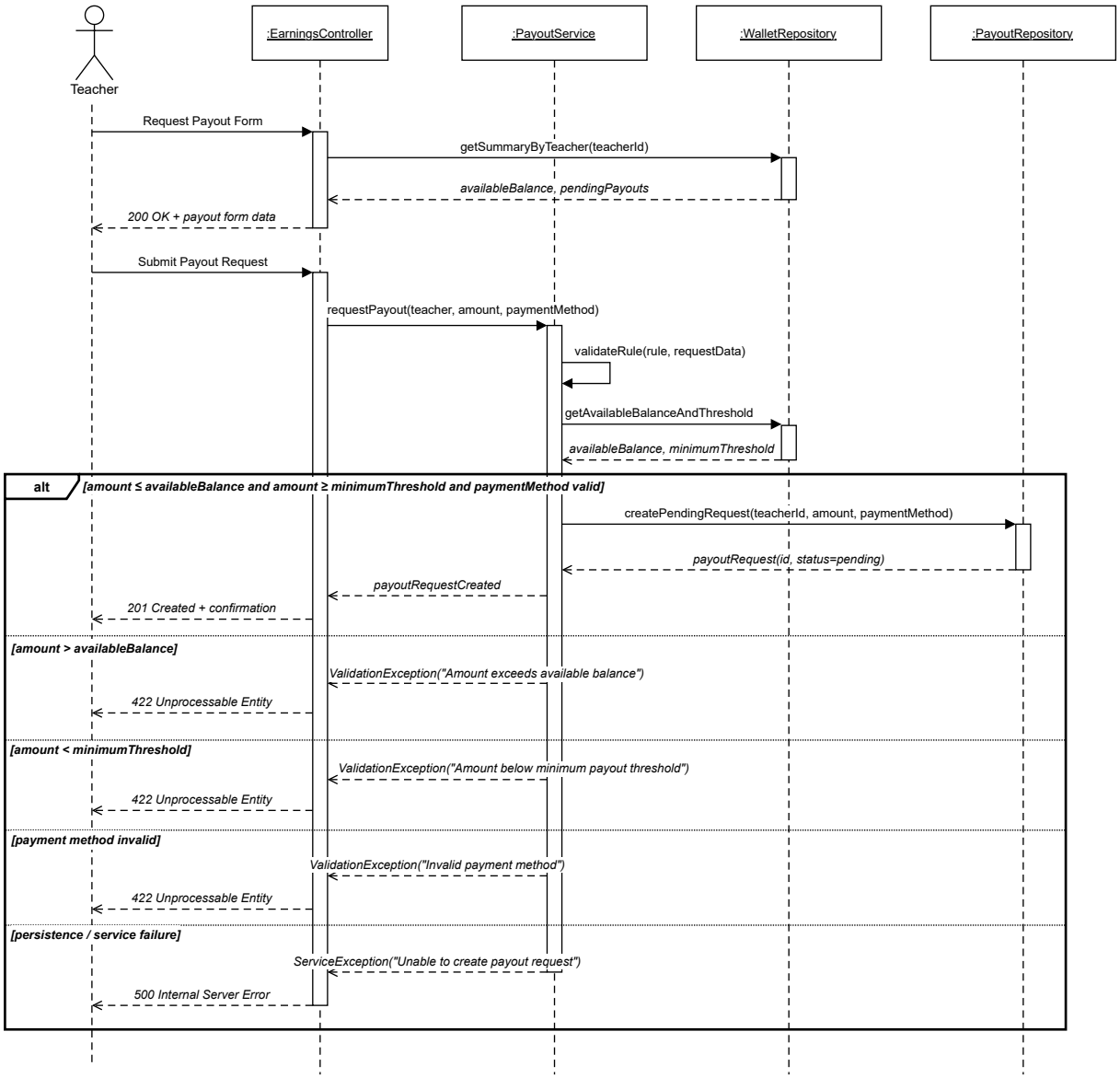

Figure 1.11: Sequence Diagram: Payout Request

4.4 Provision VM Session Sequence

Figure 1.12 illustrates the provision VM session sequence.

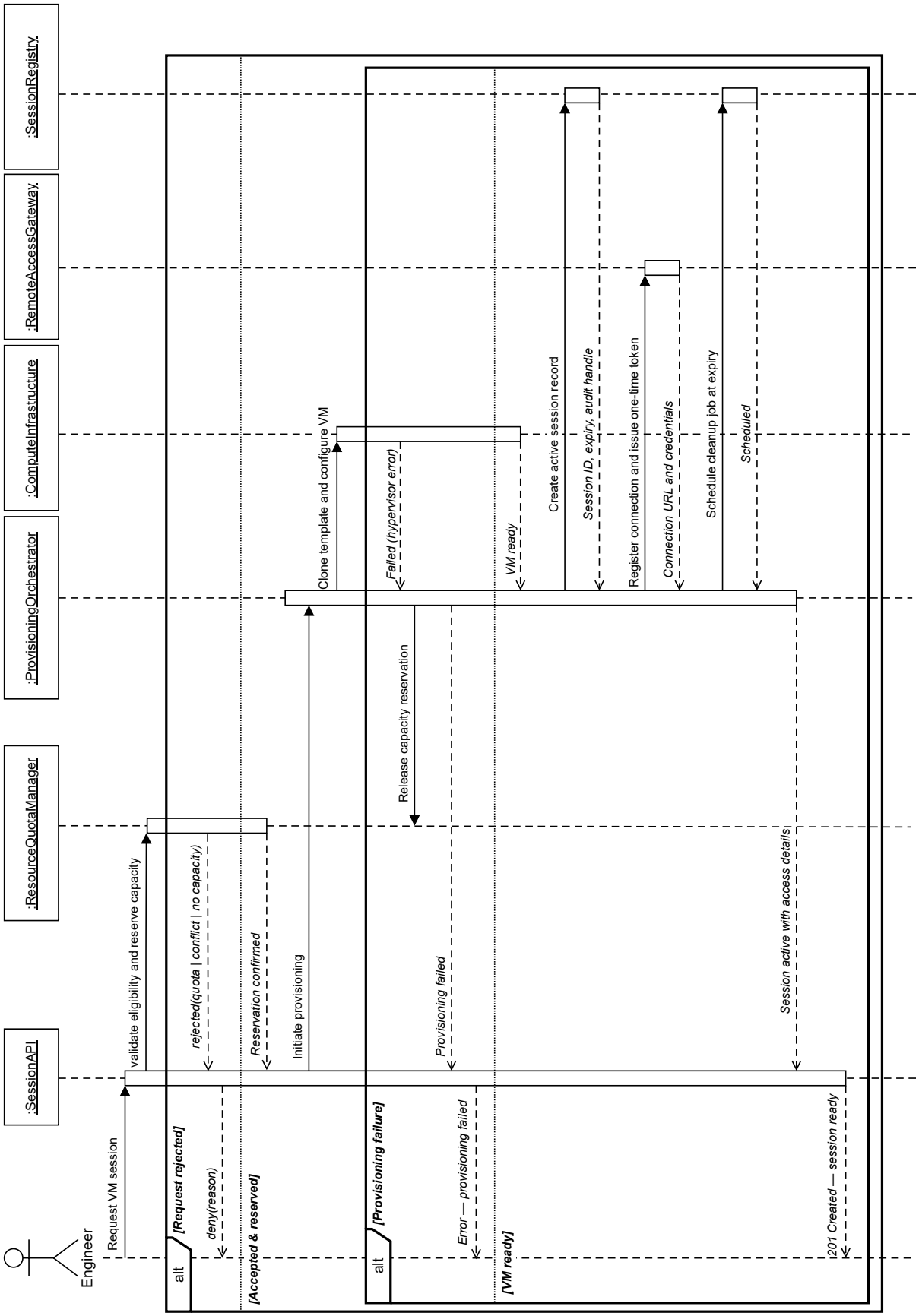

Figure 1.12: Sequence Diagram: Provision VM Session

4.5 User Authentication Sequence

Figure 1.13 illustrates the user authentication sequence.

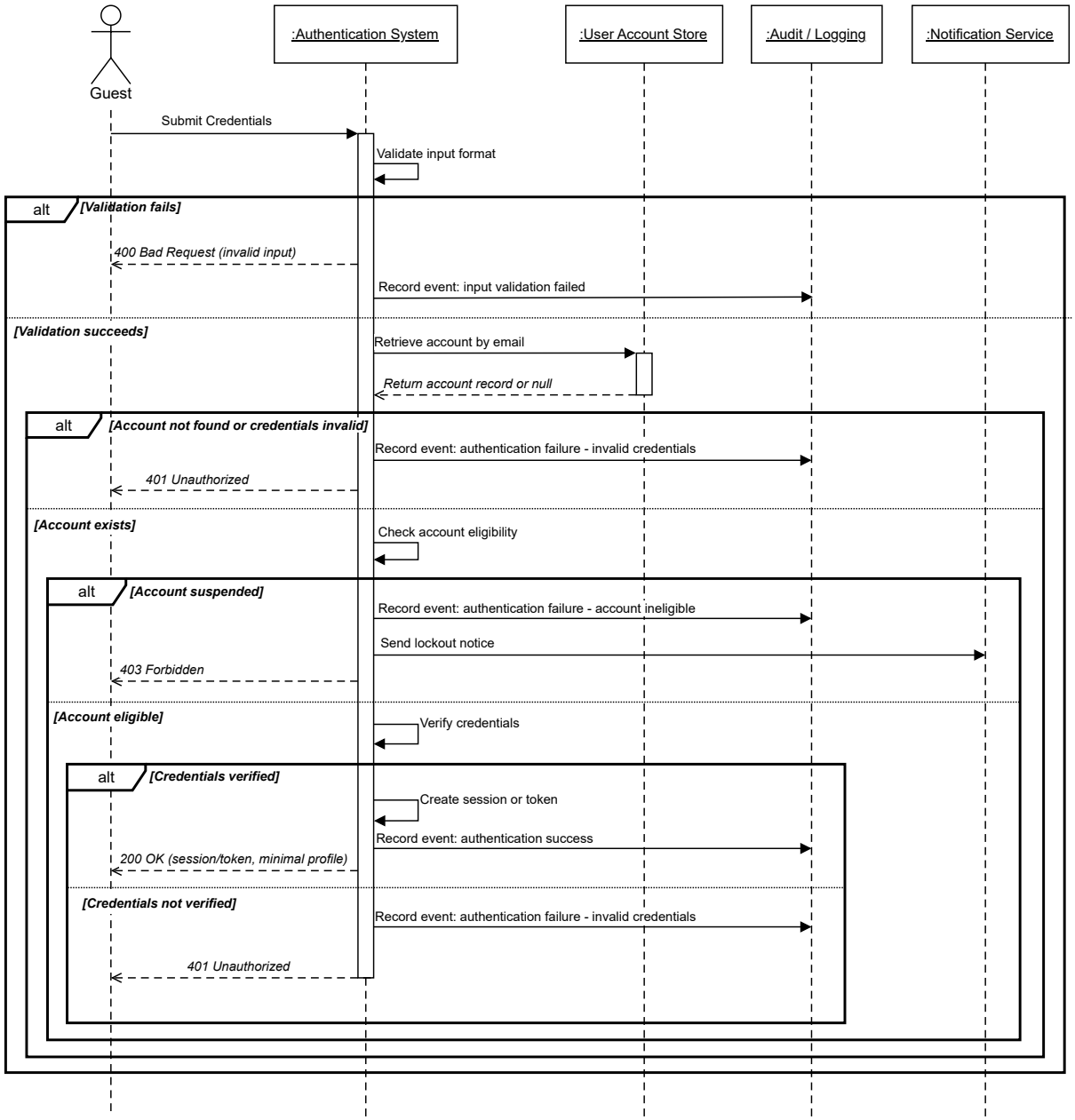

Figure 1.13: Sequence Diagram: User Authentication

Conclusion

This chapter defined the conceptual foundation of IoT-REAP through architecture, use case modeling, sequence and activity modeling, and class-level representation. These elements provide the bridge between the preliminary requirements and the realization of the platform.

Chapter 2

Realization and Experimentation

Introduction

This chapter presents the practical realization of the IoT-REAP platform. It describes the development environment, the technologies and tools used, and the principal interfaces implemented for public users, learners, teachers, engineers, and administrators.

1 Tools and Development Environment

The implementation of IoT-REAP required a development environment capable of supporting backend web application development, reactive frontend interfaces, virtual machine orchestration, and industrial remote access workflows. The working environment therefore combined hardware resources suitable for local development and testing with a modern software stack centered on Laravel, React, TypeScript, and MySQL.

1.1 Hardware Environment

The hardware environment used for IoT-REAP development was organized around three complementary layers: the developer workstation, the application execution environment, and the remote infrastructure targeted by the platform. The development workstation had to be powerful enough to run a local PHP application server, a Node.js asset pipeline, a relational database, code editors, and testing tools simultaneously. In addition, the project domain required an infrastructure context involving virtual machines, gateway nodes, cameras, and remotely accessible lab equipment.

Table 2.1: Hardware Environment for IoT-REAP Development and Execution

Hardware Element	**Role in the Project**
Developer Workstation	Used for source code editing, frontend compilation, backend execution, database interaction, and local testing of the Laravel and React application.
Application Server Environment	Hosts the Laravel application, queue workers, scheduled tasks, and the built frontend assets required by the platform.
Database Server	Stores users, training paths, training units, quizzes, payments, notifications, reservations, sessions, and other persistent business data.
Virtualization Infrastructure	Provides the virtual machines launched for hands-on practical sessions and browser-based remote access scenarios.
Remote Access Infrastructure	Includes services used to expose clientless access to provisioned lab machines and remote training environments through the browser.
Industrial and IoT Equipment	Represents the target equipment domain of the platform, including cameras, hardware gateways, USB devices, and operational training resources.

1.2 Software Environment

The software environment brought together development tools, dependency managers, testing utilities, and collaboration platforms. Since IoT-REAP is implemented as a Laravel application using Inertia.js and React, both PHP and JavaScript tooling were required throughout the development cycle.

Table 2.2: Software Development Environment

Name	Category	Role in the Project
Visual Studio Code	Code Editor	Used as the main development environment for editing PHP, TypeScript, React, route definitions, configuration files, and LaTeX report sources.
Laravel Herd	Local Server	Used as a lightweight local development environment for running the Laravel application, PHP runtime, and MySQL database during implementation.
Git and GitHub	Version Control	Used to manage source code history, feature evolution, collaboration, and recovery of stable project states during iterative development.
Composer	PHP Dependency Manager	Used to install and manage Laravel framework packages and backend dependencies such as Fortify, Inertia Laravel, Socialite, and Stripe integration libraries.
NPM	JavaScript Dependency Manager	Used to manage frontend dependencies including React, TypeScript, Vite, Tailwind CSS, Radix UI components, Zustand, React Query, and Vitest.
Vite	Build Tool	Used for frontend asset bundling, hot module replacement in development, and production asset compilation.
MySQL	Database System	Used for relational data persistence, including authentication data, training content, reservations, operational sessions, and administrative records.
DBeaver	Database Management	Used for visual database inspection, query execution, schema browsing, and data verification during development and debugging.
Docker	Containerization	Used for running containerized infrastructure services such as Mosquitto MQTT broker during development and testing.
Postman / API Testing Tools	API Validation	Useful for verifying HTTP routes, request payloads, responses, authentication behavior, and external integration endpoints during development.
PHPUnit	Backend Testing	Used to verify backend logic, route behavior, business rules, and Laravel feature flows.
Vitest	Frontend Testing	Used to test React interface logic and

1.3 Technology Stack

The IoT-REAP platform uses a hybrid web architecture combining a Laravel backend with an Inertia.js-driven React frontend. This approach offers server-side routing and domain organization on the backend while preserving a modern single-page application experience for users interacting with the platform.

Table 2.3: Core Technology Stack of IoT-REAP

Technology	Version / Type	Role in the Platform
PHP	8.2+	Main backend programming language used to implement controllers, services, models, jobs, events, policies, and platform business logic.
Laravel	12	Backend framework used for routing, authentication, queue processing, validation, authorization, database access, and modular organization of application logic.
Inertia.js	2.x	Connects Laravel routes to React pages without building a separate REST-only frontend, enabling a smooth single-page navigation model.
React	19	Used to build the interactive frontend interfaces for training paths, dashboards, teaching pages, administration panels, and user settings.
TypeScript	5.x	Provides type safety across the frontend codebase, improving maintainability and reducing interface integration errors.
Vite	7.x	Used for modern asset compilation and development server support.
Tailwind CSS	4.x	Used for responsive and utility-based user interface styling across public, learner, instructor, and administration pages.
MySQL	Relational DBMS	Stores structured application data such as users, reservations, certificates, quizzes, notifications, sessions, and payments.
Laravel Fortify	Authentication Package	Used to implement login, registration, password reset, email verification, and two-factor authentication flows.
Stripe	Payment Integration	Supports checkout, payment tracking, refund workflows, and payout-related administrative features.
Proxmox VE Integration	Infrastructure Service	Supports VM lifecycle management for practical remote lab sessions and browser-accessed training environments.
Apache Guacamole Integration	Remote Access Service	Enables clientless browser-based remote desktop access to virtual machines without requiring local desktop client installation.
Vitest and PHPUnit	Test Frameworks	Support verification of frontend behavior and backend feature logic.
Laravel Socialite	OAuth Package	Provides Google OAuth authentication, allowing users to register and log in through

1.4 Programming Languages

The platform relies mainly on PHP for backend development, TypeScript for frontend application logic, SQL for relational database interaction, and shell or command-line tooling for build and deployment operations. PHP is used inside Laravel controllers, services, models, jobs, policies, and validation classes. TypeScript is used with React to build typed and maintainable user interfaces. SQL remains present through migrations, queries, and schema-level constraints managed by Laravel's database layer.

1.5 Development Tools

The main tools used during realization include Visual Studio Code, Git and GitHub, Composer, NPM, Vite, DBeaver, Docker, PHPUnit, Vitest, Draw.io, and API testing tools. Together, these tools supported code editing, dependency management, database inspection, frontend compilation, test execution, diagram preparation, and iterative verification.

2 Implementation

This section describes the principal interfaces and functional areas implemented in IoT-REAP. The objective is not merely to present isolated screens, but to explain how the platform organizes the complete user experience across public access, technical learning, VM-based practice, teaching workflows, and administrative control.

2.1 Platform Architecture Pattern

The IoT-REAP backend follows a strict layered architecture that separates concerns across six distinct layers. Every HTTP request passes through the same lifecycle: a FormRequest validates and authorizes the input, a thin Controller delegates to a single Service method, the Service contains all business logic and dispatches events or jobs, the Repository encapsulates all database queries, the Eloquent Model defines relationships and casts, and an API Resource shapes the JSON response while hiding internal fields. This pattern is applied consistently across all 47 controllers, 54 services, 36 repositories, 45 models, 66 FormRequests, and 37 API resources that constitute the backend.

The event-driven pipeline complements the layered architecture by decoupling side effects from the request cycle. When a VM session is created, the service dispatches a VMSessionCreated event, which triggers the ProvisionVMJob to begin the Proxmox clone operation. Once the VM is running, the VMSessionActivated event fires, and the CreateGuacamoleConnectionListener

responds by establishing the remote desktop connection. Similar patterns connect certificate issuance to notification delivery, video transcoding completion to status updates, and USB device attachment progress to session state changes. In total, the platform defines 8 domain events, 3 listeners, and 7 queue jobs.

Status fields throughout the platform are modeled as strongly-typed PHP enums rather than raw strings, ensuring compile-time safety and preventing invalid state transitions. The codebase defines 21 enums covering domains such as VMSessionProtocol, PaymentStatus, CameraStatus, QuizQuestionType, TrainingPathStatus, UserRole, and VideoStatus. Error handling follows a domain exception hierarchy with 7 custom exception classes—including ProxmoxApiException, GuacamoleApiException, CameraControlConflictException, and NoAvailableNodeException—that allow controllers to map specific failure conditions to appropriate HTTP responses without inspecting error messages.

2.2 Middleware and Request Pipeline

Five custom middleware enforce cross-cutting concerns across all platform routes. The EnsureRole middleware verifies that the authenticated user possesses the required role before allowing access to role-specific routes. The RedirectBasedOnRole middleware performs intelligent post-authentication routing, directing users to their appropriate dashboard based on their role and approval status. The SecurityHeaders middleware injects CORS headers and security-related HTTP headers into every response. The HandleAppearance middleware manages user theme preferences, and the HandleInertiaRequests middleware shares common data—such as the authenticated user, flash messages, and feature flags—with every Inertia.js page render.

2.3 Landing Page

The landing page serves as the public entry point of IoT-REAP. It introduces the platform as an Industry 4.0 remote operations and technical training environment, highlighting its major capabilities such as virtual lab environments, hands-on operations training, remote industrial control, and OT-oriented security. It also presents selected featured training paths to encourage learner onboarding and provides direct navigation toward registration, login, and the main training catalog.

From an implementation perspective, this page is rendered through the React frontend using Inertia.js and receives featured training path data from the backend. It functions both as a communication interface and as a gateway into the protected parts of the platform. The platform also provides legal pages—a privacy policy and terms of service—accessible from the footer navigation, ensuring compliance with data protection and platform usage disclosure

requirements.

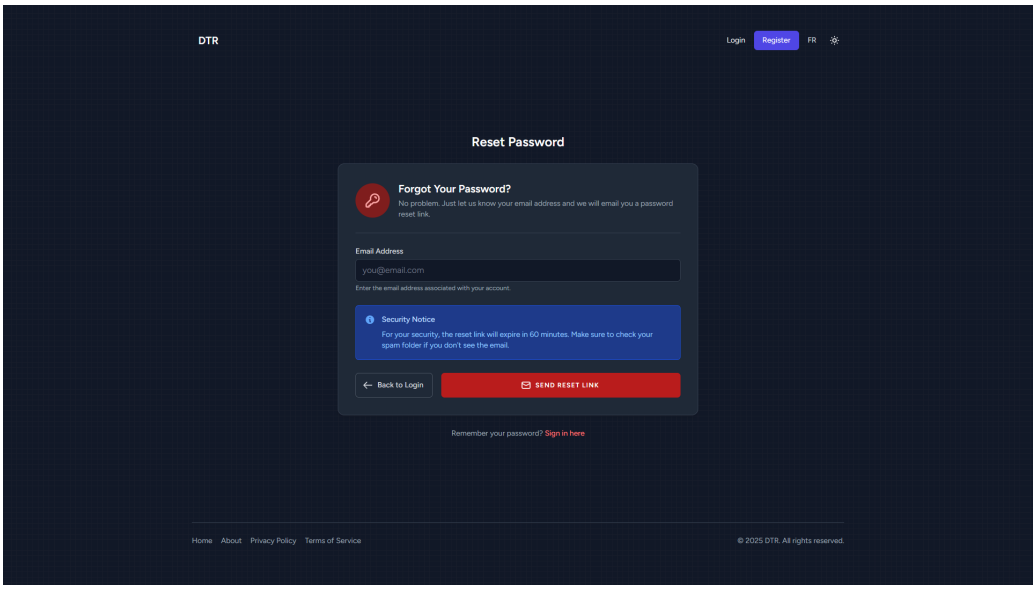

Figure 2.1: IoT-REAP Landing Page

2.4 Authentication System

The authentication system was implemented using Laravel Fortify and integrated frontend pages. It provides the core security flows required by the platform, including account registration, login, password reset, email verification, and optional two-factor authentication. This subsystem ensures that only authenticated and verified users can access sensitive areas such as lab sessions, training progression, instructor tools, and administrative operations.

A notable aspect of the implementation is role-aware redirection. Once authenticated, users are directed to different areas according to their role and approval status. Teachers may be redirected to their teaching dashboard or to a pending approval page, administrators are redirected to administration interfaces, and engineers or learners access the operational dashboard and training interfaces relevant to their permissions.

The Google OAuth flow is managed by a dedicated GoogleOAuthService and a five-endpoint controller that handles the complete lifecycle: redirect to Google, handle the callback, present a role selection page for new users, process the selected role, and complete registration. After Google authentication, users who do not yet have a platform account are presented with an oauth-select-role page where they choose their intended role. The role-aware redirect logic then directs approved teachers to the teaching dashboard, pending teachers to an approval waiting page, administrators to the admin panel, and engineers or learners to their respective operational interfaces.

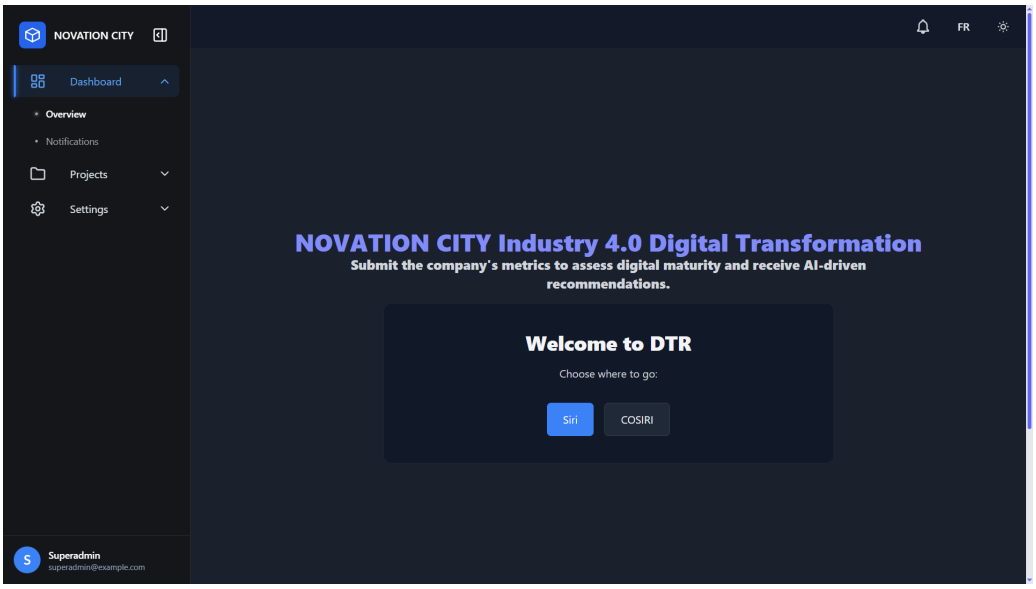

Figure 2.2: Authentication Login Interface

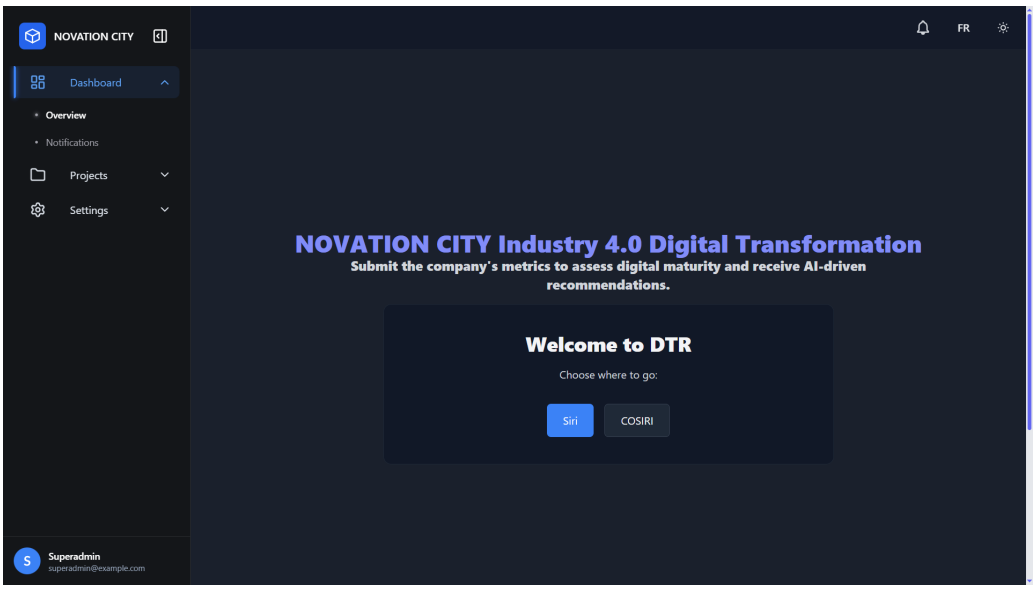

Figure 2.3: User Registration Interface with Role Selection

2.5 Operations Dashboard and VM Session Interfaces

One of the central implementation goals of IoT-REAP is the delivery of browser-accessible operational sessions. For this reason, the platform includes a dedicated session management domain through which authorized users can browse available VM resources, provision sessions, extend active sessions, terminate them, and inspect remote access details.

The VM session module interacts with Proxmox-related services to coordinate virtual machine provisioning and with Guacamole-related services to generate browser-compatible remote access tokens. This design allows the user to access training or operational

resources directly from the web application without installing a dedicated client. Additional configuration support is provided through connection preference management, allowing protocol-specific profiles and per-VM defaults to be stored. Connection preference management is implemented as a dedicated feature with its own page, controller, and two supporting models: GuacamoleConnectionPreference stores protocol-specific parameters such as color depth, resolution, and keyboard layout for RDP, VNC, and SSH protocols, while UserVMConnectionDefaultProfile stores per-VM default profiles that are automatically applied when a user connects to a specific session. Users configure these preferences through a dedicated interface accessible from the session view, and the settings are validated through dedicated FormRequest classes before being persisted.

The implementation also includes session-level resource interaction. Users can access session cameras, request control over available devices, change stream resolution, and manage hardware attachment flows. In this way, the session interface is not limited to remote desktop access; it becomes a unified work area for virtual and physical resource usage.

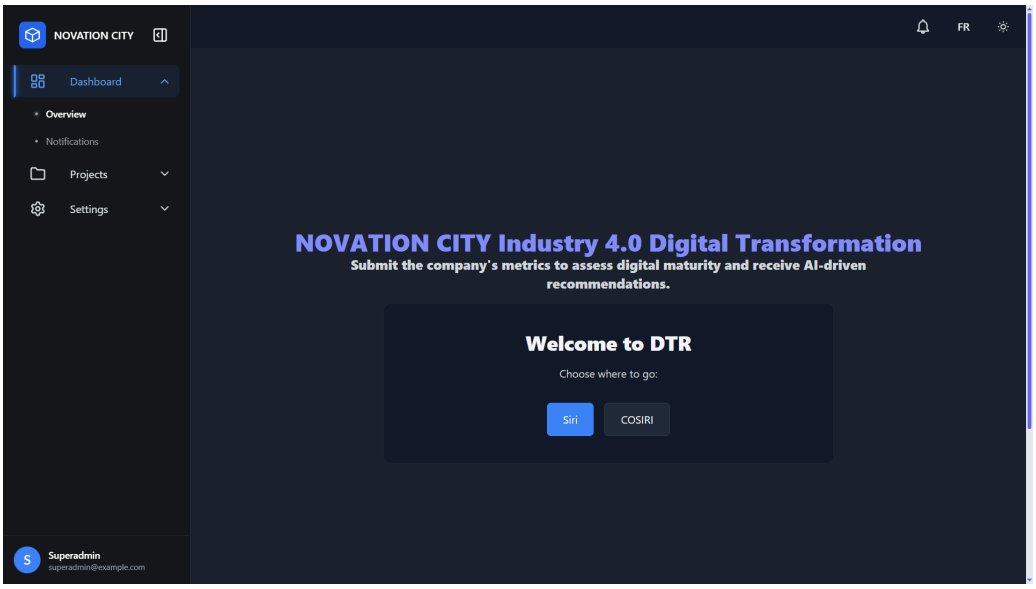

Figure 2.4: VM Session Management and Remote Access Interface

The backend implementation of the VM session domain follows the platform's layered architecture. The VM session creation service receives validated input from the controller, persists the session record through the repository layer, and then delegates Guacamole connection setup to the Guacamole client service. The service logs each step of the process—session creation, connection establishment, and error conditions—to support operational monitoring and debugging. If the Guacamole connection fails, the exception is caught, logged with contextual details, and re-thrown so that the controller can return an appropriate error response.

The Guacamole client service handles the creation of browser-accessible remote desktop

connections through the Guacamole HTTP API. It first obtains an authentication token from the Guacamole server, then separates connection parameters (such as hostname, port, and credentials) from connection attributes (such as maximum concurrent connections and failover settings). The service sends a structured request to the Guacamole API endpoint, including the connection name, protocol identifier, parent folder, parameters, and attributes. Upon success, the API returns a unique connection identifier that the service stores in the session record for later retrieval.

On the frontend, the VM session page uses a custom React hook that encapsulates all data fetching and session lifecycle actions. The hook manages three state variables: the list of active sessions, a loading indicator, and an error message. When the component mounts, the hook automatically fetches the user's sessions from the backend API. It also exposes functions for creating new sessions and terminating existing ones, both of which trigger a refresh of the session list after completion. This pattern ensures that the UI always reflects the latest session state without requiring manual data synchronization.

The API resource layer transforms Eloquent models into consistent JSON responses consumed by the React frontend. Each session resource includes the session identifier, current status, protocol type, connection profile name, VM identifier, node name, expiration timestamp, remaining time in seconds, IP address, Guacamole connection identifier, and the Guacamole access URL. When the user relationship is loaded, the resource also includes the user's identifier, name, and email. This structured response format ensures that the frontend receives only the data it needs, while internal fields such as Proxmox credentials and Guacamole tokens remain excluded.

The Proxmox integration includes a retry mechanism with exponential backoff: the provisioning service attempts the clone operation up to three times with increasing delays (2, 5, and 10 seconds), handling transient API failures gracefully. Template VMs are allocated from the VMID range 100–199, while session VMs receive identifiers from the range 200–999, preventing conflicts between templates and running instances. The ProxmoxLoadBalancer selects the target node using a weighted scoring algorithm that prioritizes available RAM at 70 percent weight over CPU utilization at 30 percent, with a load threshold of 85 percent to avoid overloading any single node. When a VM is first created, the ProxmoxIPResolver actively polls the guest agent for the assigned IP address, falling back to a static DHCP lease assignment if the agent does not respond within the timeout period.

Sessions support two operational modes: ephemeral sessions that are automatically destroyed upon expiration, and persistent sessions whose VMs are preserved for repeated access. Users can extend active sessions through the ExtendSessionService, and the VMReservationService allows advance booking of VM resources with conflict detection. The platform also supports VM snapshot operations—create, delete, and rollback—enabling users to save and restore VM states during practical exercises.

2.6 Training Path Interfaces

The training path module transforms IoT-REAP from a pure remote access platform into a complete technical learning environment. Public users can browse training paths, read details, inspect reviews, and search by category or keyword. Authenticated learners can enroll, access their personalized list of training paths, and continue their progression through assigned modules and training units.

From an implementation perspective, this module combines public catalog features with learner-specific progression tracking. The backend manages business rules related to enrollment, completion, review management, and note aggregation, while the frontend provides the structured navigation and responsive display needed for an educational workflow.

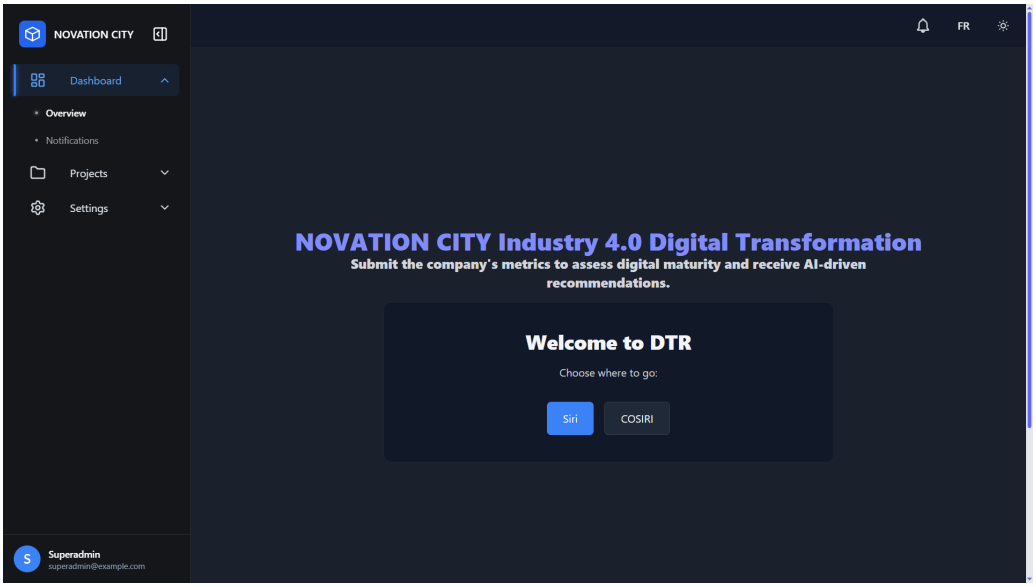

Figure 2.5: Training Path Catalog and Learner Progression Interface

2.7 Training Unit Learning Interface

The training unit interface is the operational center of the learner experience. Each training unit can be associated with different instructional media and practical resources such as videos, reading articles, quizzes, notes, and VM-based activities. This design supports blended learning, where theoretical content and practical execution coexist within the same structured path.

Video-oriented units support streaming and progress tracking, article units support reading completion, quiz units support attempt management and grading history, and VM-enabled units can be linked to remote lab resources. This implementation directly supports the pedagogical objective established in the previous chapters: enabling technical upskilling through guided learning combined with hands-on practice.

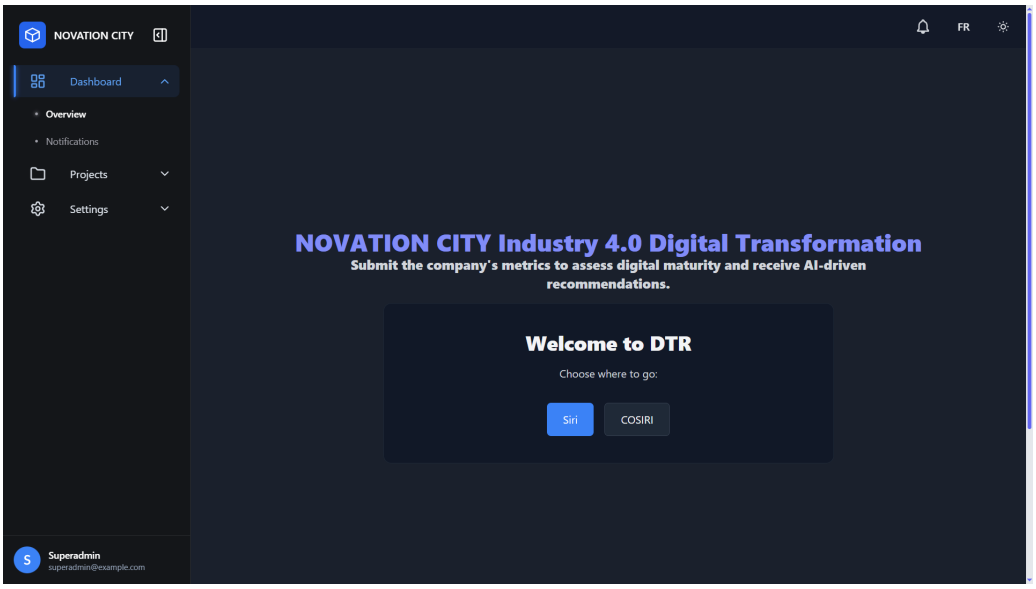

Figure 2.6: Training Unit Learning Interface with Blended Content

2.8 Teaching Interfaces

IoT-REAP includes a complete teaching area for instructors responsible for creating and maintaining technical content. The teaching module allows instructors to create training paths, update path metadata, define modules, create and reorder training units, and submit completed content for administrative review. This authoring workflow makes the platform extensible and enables internal growth of the training catalog.

The implementation goes further by providing specialized content editors for quizzes, articles, and videos. Instructors can configure quiz questions, publish or unpublish assessments, upload or manage training videos, maintain captions, and edit article-based lesson content. In addition, the platform includes instructor analytics, learner progress monitoring, earnings visualization, payout requests, and a forum inbox for handling learner interactions. This makes the teaching area not only a content management space but also an operational dashboard for pedagogical supervision.

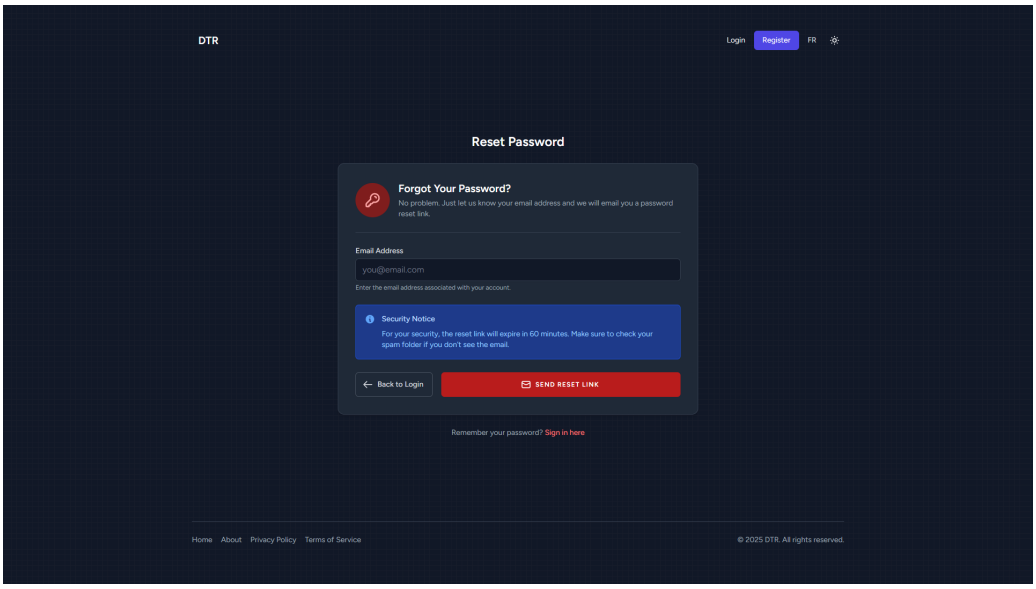

Figure 2.7: Instructor Teaching Dashboard and Content Authoring Interface

2.9 Administrative Interfaces

The administrative domain is one of the broadest implementation areas of the platform. It supports governance of infrastructure, users, content approval, reservation workflows, financial operations, and operational supervision. Administrators can access a central dashboard with analytics and health indicators, manage infrastructure records, inspect nodes and virtual machines, configure Proxmox server records, and oversee hardware and maintenance states.

The administrative features also include user management with approval, suspension, role updates, impersonation, and privacy-related deletion actions. Content governance is supported through training path approval and featuring workflows. Operational control includes reservation review, camera assignment and reservation management, refund handling, payout processing, alert monitoring, activity logging, and moderation of flagged forum content.

This breadth of implementation reflects the dual nature of IoT-REAP: it is simultaneously a learning management platform and an operational orchestration system for remote industrial training resources.

Operational auditability is supported through an ActivityLogService that records significant platform events—such as user approvals, role changes, content publications, and administrative actions—in a persistent activity log. Administrators can query and filter this log through a dedicated controller, providing a traceable record of governance actions. The platform also maintains a SystemAlert model and associated AdminAlertService and SystemHealthService that track infrastructure health indicators, VM provisioning failures, and operational anomalies. Alerts are displayed on the administrative dashboard and can trigger notifications to responsible administrators.

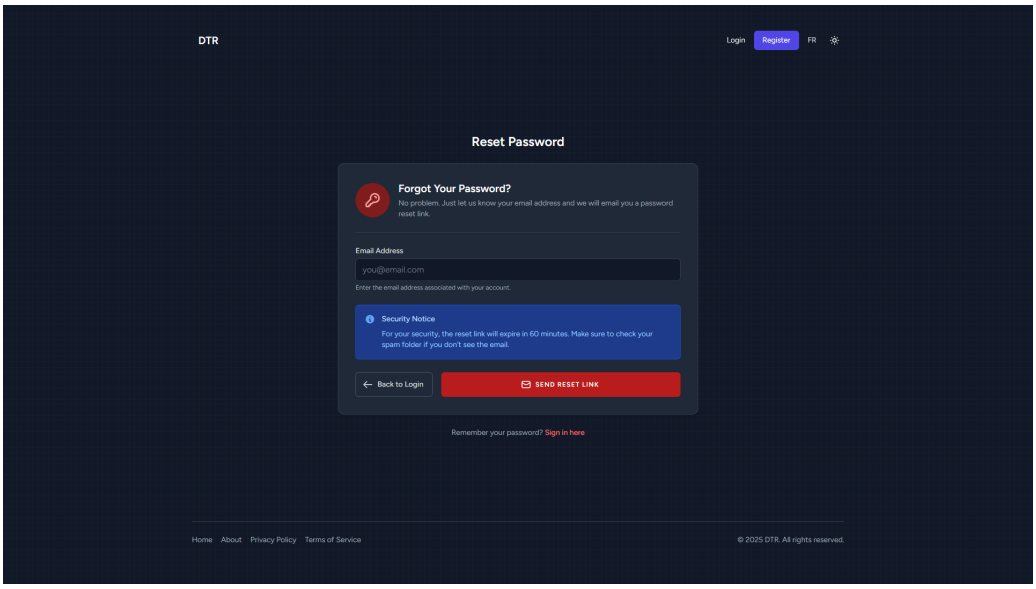

Figure 2.8: Administrative Dashboard and Infrastructure Governance Interface

2.10 Notifications, Certificates, Payments, and Reservations

Several cross-cutting features complete the functional landscape of the platform. These areas—notifications, certificate generation and verification, payment and checkout, and reservation management—are described in the following dedicated subsections. Each subsection below provides the implementation details and operational considerations for that specific cross-cutting concern.

2.11 Forum and Learner Interaction

To support collaborative learning, IoT-REAP integrates a forum module where users can view, create, reply to, and interact with discussion threads attached either to training paths or to individual training units. This feature encourages peer assistance, content clarification, and knowledge sharing around technical training materials.

The implementation includes user interaction features such as upvoting, flagging, and reply management, while moderators and instructors receive additional controls for pinning threads, locking discussions, resolving flags, and marking helpful answers. This makes the forum both a learning tool and a moderated support space.

2.12 Camera and Robotics Interface

The camera and robotics interface is a distinctive feature of IoT-REAP that connects the web platform to the physical world of industrial and IoT equipment. This module provides

browser-based access to live camera streams from operational environments and allows authorized users to issue remote commands to connected robots and devices.

From the backend perspective, camera management is handled through dedicated services that register camera devices, assign them to infrastructure nodes, and manage their availability for reservation. The streaming pipeline uses a WHEP (WebRTC-HTTP Egress Protocol) proxy for browser-compatible live stream delivery, with FFmpeg handling format adaptation and transcoding when required for browser playback.

Camera pan-tilt-zoom (PTZ) operations are protected by an exclusive lock mechanism: only one user may hold the PTZ lock for a given camera at a time, preventing conflicting control commands from multiple simultaneous users. Camera visibility is scoped to the user's role and session context—engineers can only view and control cameras assigned to their active sessions, while administrators have global visibility. The platform supports multiple camera types including fixed IP cameras, PTZ-capable cameras, and USB cameras attached through gateway nodes, each with type-specific configuration parameters stored in the database.

The robotics command interface uses MQTT as the communication protocol. Robot commands are published to topic-based channels following a robot-specific command topic format with QoS 1 delivery guarantees, ensuring that operational instructions reach their target devices reliably. Telemetry data flows in the opposite direction, with devices publishing to a robot-specific telemetry topic and the platform subscribing to receive status updates. This bidirectional communication model enables real-time remote operation of industrial equipment from within the browser.

The MQTT service implementation demonstrates how the platform publishes robot commands with QoS 1 delivery. The service constructs the topic by replacing a placeholder in the configured topic pattern with the target robot identifier, then assembles a structured payload containing the action type, parameters, and a timestamp. The publish method first attempts delivery through an HTTP bridge to the gateway agent, which provides a reliable and centrally managed publishing path. If the HTTP bridge is unavailable—for example, due to a network timeout—the service logs a warning and falls back to direct CLI-based publishing. This dual-path approach ensures that robot commands are delivered even when the preferred bridge mechanism is temporarily unreachable.

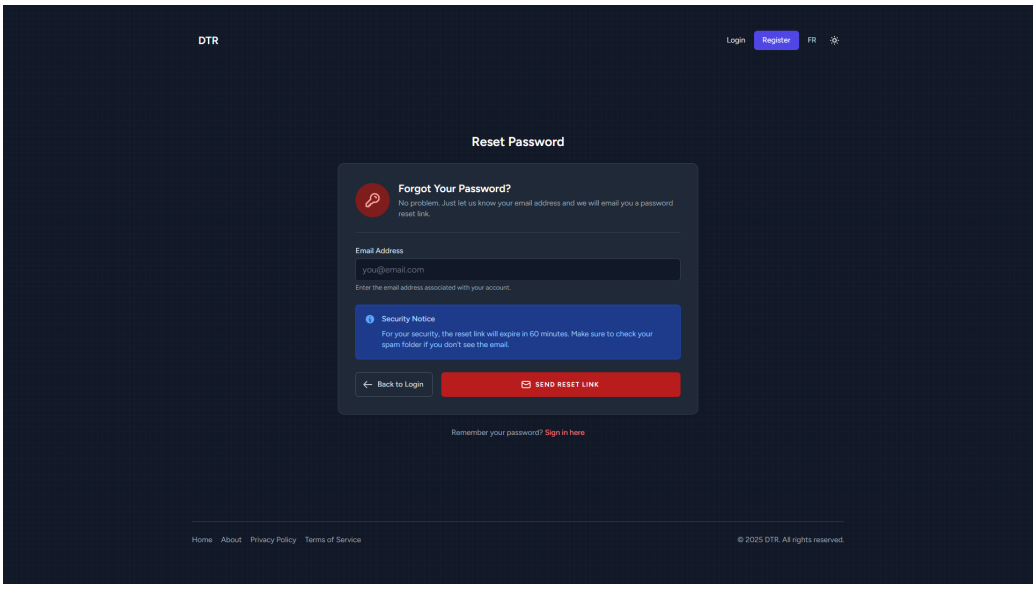

Figure 2.9: Camera and Robotics Control Interface

2.13 Hardware and USB/IP Interface

The hardware and USB/IP interface extends IoT-REAP's remote access capabilities beyond virtual machines to physical USB devices attached to gateway nodes. This module enables authorized engineers to attach remote USB devices—such as cameras, serial adapters, or programming tools—to their active VM sessions over IP, effectively providing the same experience as if the device were physically plugged into their local machine.

The implementation relies on USB/IP protocol services running on gateway nodes, managed through a dedicated backend service that tracks available devices, their connection states, and their assignment to active sessions. When a user requests a USB device attachment, the platform coordinates the USB/IP binding on the target gateway node and updates the session record to reflect the hardware state. Device detachment and cleanup are handled automatically when sessions expire or are terminated, preventing orphaned device locks.

Administrators can manage the hardware inventory, register new USB devices, assign them to specific gateway nodes, and monitor their utilization across sessions. This interface is essential for scenarios where engineers need to interact with physical training equipment—such as microcontrollers, PLCs, or industrial sensors—during remote lab sessions.

The USB attachment workflow uses a queued job system to handle the potentially slow USB/IP binding process asynchronously. Device paths on Windows gateway nodes are resolved using DOS 8.3 short-name format to avoid issues with spaces and special characters in USB device paths. An auto-reattach mechanism monitors active sessions and reattaches USB devices that were previously bound if the VM is restarted or migrated. A reconcile artisan command is also available to scan all gateway nodes, compare reported USB devices against the database

inventory, and correct any drift between the actual hardware state and the recorded state.

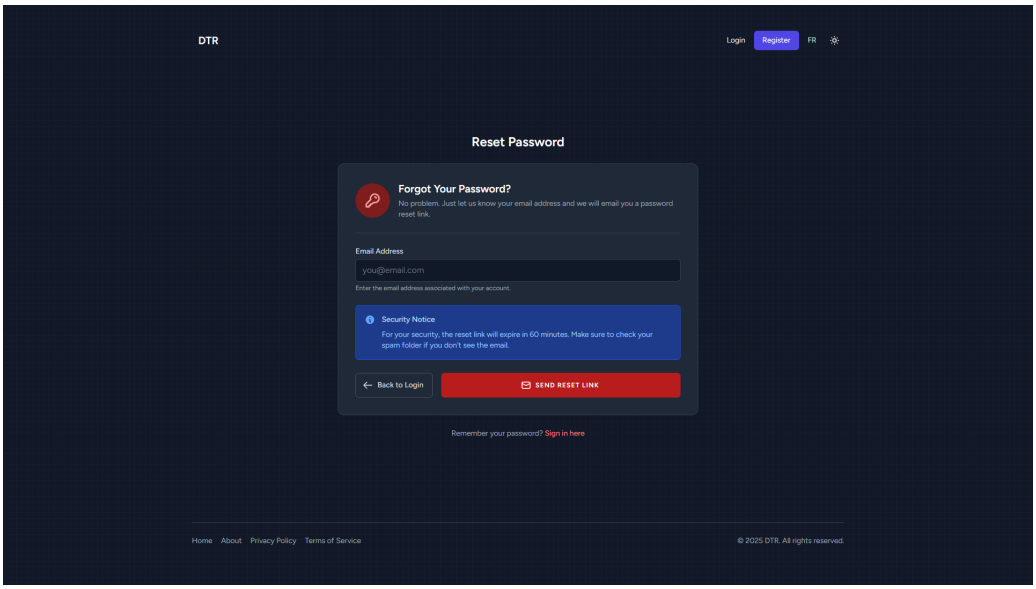

Figure 2.10: Hardware and USB/IP Device Management Interface

2.14 Search and Discovery Interface

The search and discovery module provides users with efficient ways to find training paths, training units, and other platform content. The implementation supports keyword-based search with filtering by category, difficulty level, and content type, enabling learners and visitors to quickly locate relevant educational resources.

From the backend, search queries are processed through a dedicated SearchRepository that applies scoped queries against training path and training unit records, with the SearchService coordinating query construction and result ranking. The frontend exposes the search interface through a GlobalSearch component that provides a command-palette-style interface with instant results, category badges, and direct navigation links. This module is particularly important for platforms with growing catalogs, where manual browsing alone would be insufficient for content discovery.

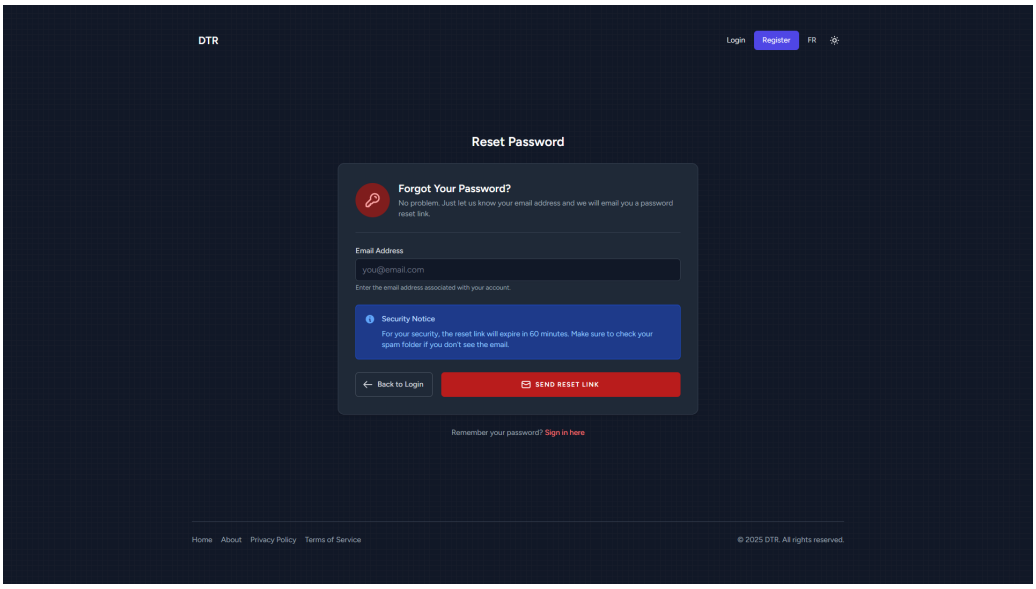

Figure 2.11: Search and Discovery Results Interface

2.15 Video Streaming Interface

The video streaming interface supports the delivery of instructional video content within training units. Unlike the live camera streams described earlier, this module focuses on pre-recorded educational videos that instructors upload and manage as part of their teaching content.

The implementation handles video upload, storage, transcoding, and progressive delivery. Uploaded videos are processed through FFmpeg for format normalization and HLS segment generation, enabling adaptive bitrate streaming across different network conditions. The frontend player supports playback controls, progress tracking, and completion reporting, which feeds back into the learner's training unit progression state.

Instructors can manage their video library, update captions, replace source files, and control publication status. The video module integrates with the broader training unit system, ensuring that video-based lessons are tracked consistently alongside other content types such as quizzes and articles.

The transcoding pipeline generates three adaptive quality profiles: 360p at 800 kbps for low-bandwidth conditions, 720p at 2500 kbps for standard connections, and 1080p at 5000 kbps for high-fidelity playback. HLS segments are produced at 10-second intervals, and a thumbnail is automatically extracted at the 5-second mark at 640 by 360 resolution. The VideoService supports three execution modes for FFmpeg: auto mode, which attempts gateway-based transcoding via SSH and falls back to local execution if the gateway is unavailable; gateway-only mode, which requires a gateway node; and local-only mode, which runs FFmpeg directly on the application server. Transcoding and thumbnail generation are

dispatched as asynchronous queue jobs—TranscodeVideoJob and GenerateThumbnailJob—to avoid blocking the request cycle. The VideoProgressService tracks each learner's playback position, enabling resume-from-last-position behavior across sessions.

Figure 2.12: Video Streaming Player within a Training Unit

2.16 Quiz Engine Interface

The quiz engine is a specialized content module that supports the creation, delivery, and evaluation of assessments within training units. Instructors can define quizzes with multiple question types, configure attempt limits and time constraints, and review grading results. Learners interact with the quiz engine to attempt assessments, view their scores, and review their answer history.

The backend implementation manages quiz lifecycle states (draft, published, closed), question ordering, answer validation, and score calculation. The quiz engine also supports automatic grading for objective question types and provides instructors with analytics on question difficulty and learner performance distribution. This module is tightly integrated with the training unit progression system, ensuring that quiz completion contributes to the learner's overall advancement.

Question types are governed by a QuizQuestionType enum that defines the supported formats: multiple-choice, true/false, and short-answer. Each quiz attempt is tracked through a QuizAttempt model that records the learner's answers, score, and timestamp, enabling both the learner and instructor to review historical performance. On the frontend, the quiz experience is delivered through dedicated React components for quiz attempt rendering, a quiz builder interface for instructors, and a results review page that displays per-question scoring and feedback.

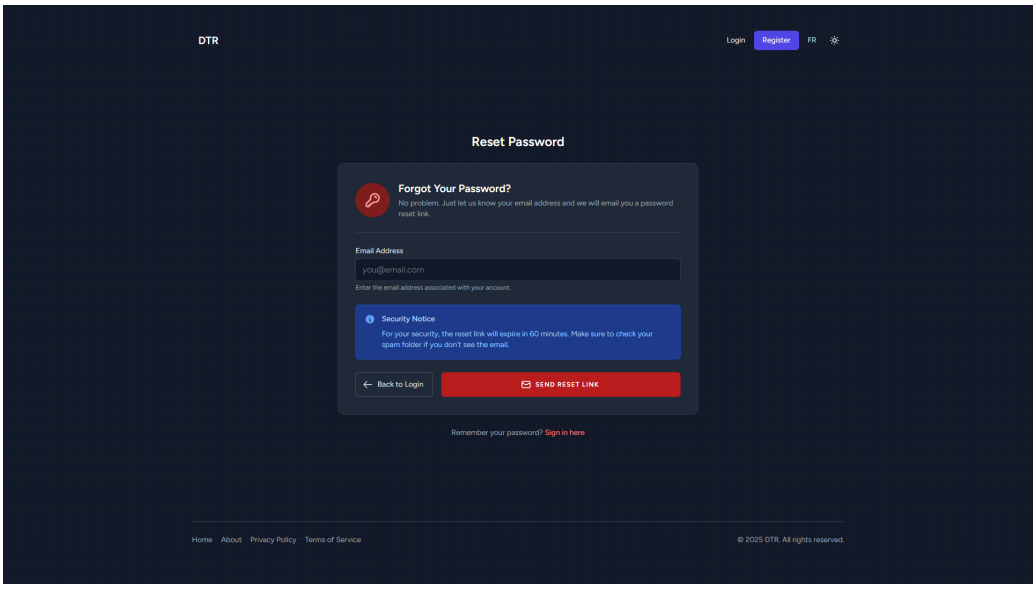

Figure 2.13: Quiz Engine Attempt and Grading Interface

2.17 Notifications Module

The notifications module provides real-time and persistent notification delivery to keep users informed about platform events relevant to their role and activity. Notifications cover a wide range of events including enrollment confirmations, session expiration warnings, payment receipts, certificate generation, content approval decisions, and forum replies.

The implementation uses a database-backed notification store with read-state tracking, allowing users to view both unread and historical notifications. The frontend renders notifications through a dedicated panel accessible from the application header, with badge indicators for unread counts. Notifications are also dispatched via email for critical events, ensuring that important information reaches users even when they are not actively using the platform.

Real-time event delivery is implemented through Laravel Reverb, a first-party WebSocket server that broadcasts platform events to connected browsers. The CustomDatabaseChannel handles notification persistence, while the broadcast authorization channels defined in the routing configuration ensure that users only receive events relevant to their sessions and roles. The platform defines nine notification types covering the principal event categories: payout approval and rejection notifications for teachers, refund approval and rejection notifications for learners, session activation failure alerts for engineers, teacher account pending approval notifications for administrators, training path approval and rejection notifications for content authors, and USB device availability notifications for engineers awaiting hardware access.

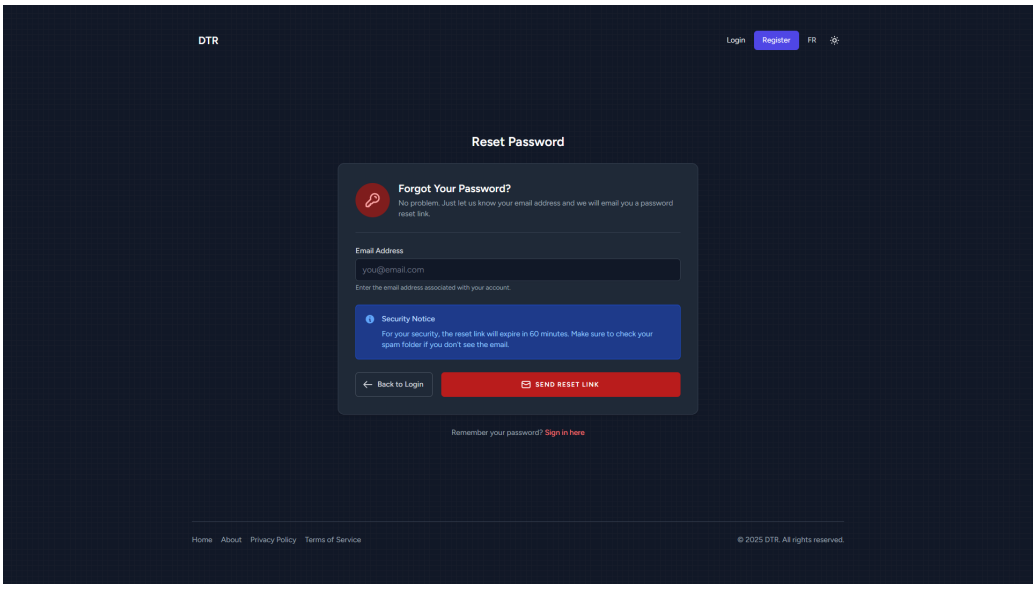

Figure 2.14: Notifications Panel

2.18 Certificate Generation and Verification

The certificate module generates formal proof of training path completion for learners who have satisfied all unit requirements. Certificates are produced as downloadable documents containing the learner's name, the training path title, the completion date, and a unique verification code.

A key implementation feature is the public verification endpoint, which allows third parties to confirm the authenticity of a certificate by entering its verification code on a public page. This mechanism adds formal credibility to the training delivered through IoT-REAP and supports professional recognition of the skills acquired.

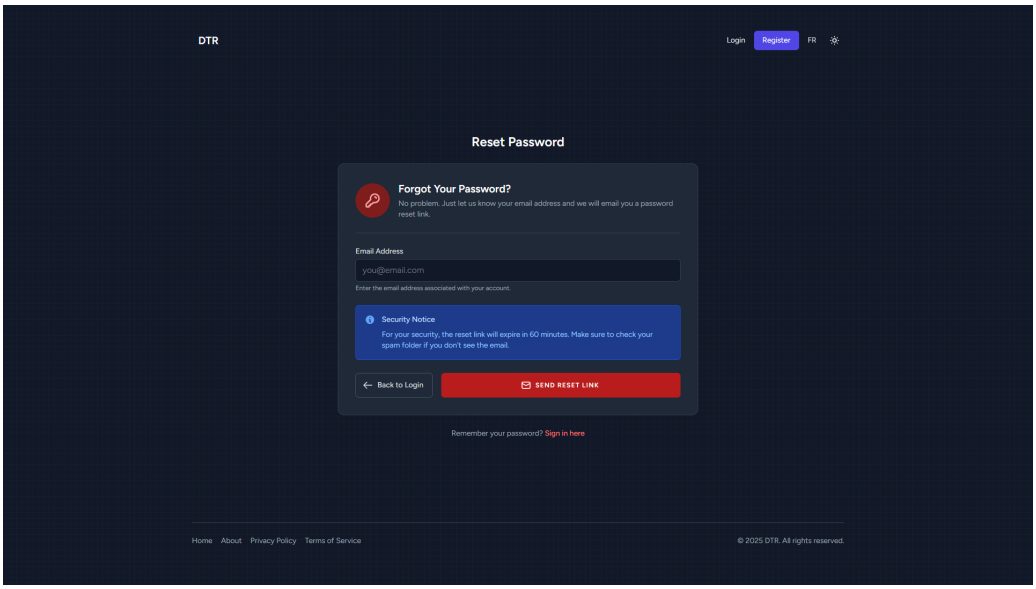

Figure 2.15: Certificate Generation and Verification Interface

2.19 Payment and Checkout Interface

The payment module integrates Stripe to handle secure financial transactions for training path enrollment. The checkout flow creates a Stripe Checkout Session, redirects the user to the Stripe-hosted payment page, and processes the webhook callback to confirm payment completion and trigger automatic enrollment.

The implementation includes webhook signature verification to prevent spoofed payment confirmations, idempotent handling of duplicate webhook events, and administrative interfaces for refund processing and payout management. Instructors can track their earnings and request payouts through the teaching dashboard, while administrators oversee the full financial lifecycle including refund approvals and payout execution.

The Stripe webhook handler illustrates how the platform processes payment completion events. When a checkout session completed event is received, the handler looks up the corresponding payment record by the Stripe session identifier. If no matching payment is found, a warning is logged and processing stops. If the payment has already been marked as completed—which can occur when Stripe sends duplicate webhook events—the handler logs an informational message and exits idempotently. Otherwise, the payment is marked as completed with the Stripe payment intent identifier, and the user is automatically enrolled in the corresponding training path. Each step is logged with contextual details to support financial auditing and operational debugging.

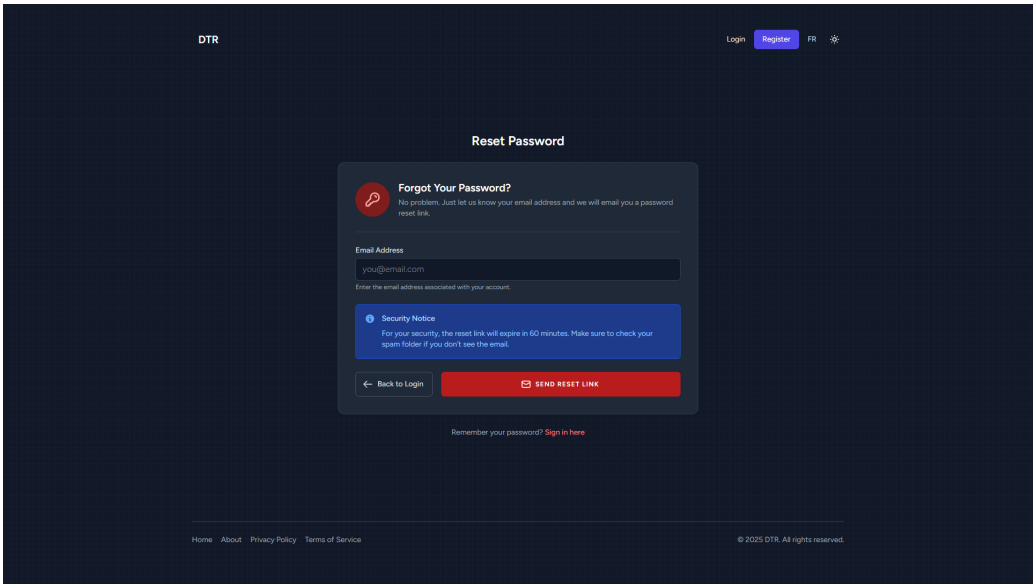

Figure 2.16: Payment and Checkout Flow

2.20 Reservation Management Interface

The reservation module enables users to book shared physical resources such as cameras and hardware devices for specific time slots. This is critical in environments where training

equipment is limited and must be shared fairly among multiple users.

The implementation includes availability checking, conflict prevention, reservation creation and cancellation, and administrative oversight for managing reservation policies. Users receive notifications about upcoming reservations and expiration reminders, while administrators can review reservation utilization patterns and adjust resource allocation accordingly.

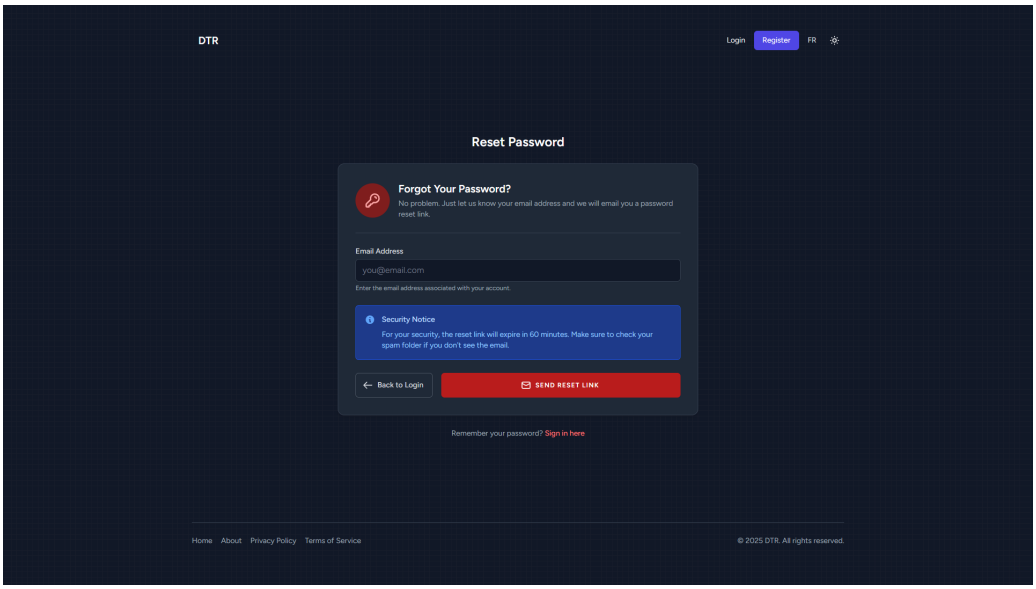

Figure 2.17: Reservation Management Interface

Conclusion

This chapter presented the technical environment and the main realized modules of IoT-REAP, including authentication, remote sessions, learning interfaces, teaching workflows, administration, reservations, video streaming, quizzes, certificates, payments, and hardware-related features.